DIABETES DETECTION USING THERMAL FOOT IMAGING AND MOBILENET-BASED DEEP LEARNING

A PROJECT REPORT

Submitted to

SRI VENKATESWARA UNIVERSITY COLLEGE OF ENGINEERING

In partial fulfilment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

CHANDRAMOULESHWAR J	**(12116013)**
KALLURI NIKITHA	**(12116030)**
MOCHARLA HARSHINI	**(12116048)**

Under the guidance of

P.HIMA CHANDANA

Academic Consultant

Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRI VENKATESWARA UNIVERSITY COLLEGE OF ENGINEERING

SRI VENKATESWARA UNIVERSITY

TIRUPATI-517502(A.P), INDIA 2024-2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRI VENKATESWARA UNIVERSITY

TIRUPATI-517502, AP, INDIA

CERTIFICATE

This is to certify that the dissertation entitled "**DIABETES DETECTION USING THERMAL FOOT IMAGING AND RESNET-BASED DEEP LEARNING**" is a bonafide work carried out by **CHANDRAMOULESHWAR J (12116013), KALLURI NIKITHA (12116030), MOCHARLA HARSHINI (12116048)** in partial fulfilment of requirements for the award of the degree of BACHELOR OF TECHNOL0GY with the specialization in COMPUTER SCIENCE& ENGINEERING, during the academic year 2024-2025.

GUIDE

P.HIMA CHANDANA,
ACADEMIC CONSULTANT,
PROJECT GUIDE
Department of C.S.E,
SVU College of Engineering,
Tirupati, 517502.

HEAD OF THE DEPARTMENT

Dr. D.VIVEKANANDA REDDY,
Professor & Head
HEAD OF THE DEPARTMENT
Department of C.S.E,
SVU College of Engineering,
Tirupati, 517502.

DECLARATION

We hereby declare that this final project titled **"DIABETES DETECTION USING THERMAL FOOT IMAGING AND MOBILENET-BASED DEEP LEARNING"** is a genuine final project work carried out by us in B.Tech Computer Science and Engineering at **Sri Venkateswara University College of Engineering, Tirupati** has not been submitted to any other course or University for the award of any degree by us.

SIGNATURE OF THE STUDENT

1.

2.

3.

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to my supervisor P.Hima Chandana, Academic Consultant Department of Computer Science and Engineering, Sri Venkateswara University College of Engineering, svu, Tirupati, for her immense help, guidance, stimulating suggestions and encouragement all the time with this thesis work. She always provided me motivating and enthusiastic atmosphere to work with, it was a great pleasure to do this thesis under her supervision.

I am deeply obliged to Dr. D.Vivekananda Reddy, Professor and Head, Department of Computer Science and Engineering, Sri Venkateswara University College of Engineering, for his constant help, encouragement and timely advice.

I acknowledge my sincere thanks to Prof .G .Sreenivasulu, Principal of S.V. University College of Engineering, SVU, Tirupati and Prof. Ch. D. V. Subba Rao, Vice Principal of S.V. University College of Engineering,SVU,Tirupati for their kind cooperation in making this thesis work a success.

I acknowledge my special thanks to all the faculty members of Department of Computer Science and Engineering, SVUCE, SVU, Tirupati, for their kind cooperation in making this thesis work a success.

I would like to express my thanks to my parents for their continued support & encouragement through the ups & downs encountered on the path to completing this thesis.

CHANDRAMOULESHWAR J (12116013)
KALLURI NIKITHA (12116030)
MOCHARLA HARSHINI (12116048)

ABSTRACT

Diabetes mellitus is a prevalent metabolic disorder that affects millions globally, leading to severe complications such as diabetic foot ulcers (DFU), which can result in infections and amputations if left undiagnosed. Traditional diagnostic methods rely on physical examinations and medical imaging, which can be invasive, time-consuming, and prone to human error. The aim of this project is to design and develop a non-invasive, AI-powered diagnostic tool that enhances early detection and monitoring of diabetic foot ulcers.

Leveraging deep learning algorithms, the system efficiently analyses thermal foot images to identify temperature variations indicative of DFUs. By employing convolutional neural networks (CNNs), particularly MobileNetV2, the model accurately classifies images into healthy and affected categories, significantly improving diagnostic precision. The system also provides real-time analytics, helping healthcare professionals make informed decisions while enabling patients to track disease progression over time.

Through its intuitive interface, both medical practitioners and patients can seamlessly interact with the platform, improving accessibility and user experience. Additionally, the system offers recommendations, predictive insights, and diagnostic analytics, aiding in personalized treatment plans and early intervention strategies. With its ability to automate detection, optimize diagnosis, and reduce healthcare burdens, this AI-powered solution represents a transformative step in medical imaging and diabetic care, ultimately enhancing patient outcomes and reducing amputation risks.

Key Words: Diabetic Foot Ulcer Detection, Deep Learning, Convolutional Neural Networks, Thermal Imaging, AI-Powered Diagnosis, Medical Analytics, Non-Invasive Screening.

TABLE OF CONTENTS

CHAPTER NO.	CHAPTER NAME	PAGE NO.
	Acknowledgements **Abstract** **Table of Contents** **List of Figures**	iv v vi-vii viii
1.	**CHPTER 1: INTRODUCTION**	**2-8**
	1.1 Motivation	2
	1.2 Problem Statement	3-4
	1.3 Objective of the Project	5
	1.4 Scope	6
	1.5 Project Overview	7-8
2.	**CHAPTER 2: LITERATURE SURVEY**	**9-14**
	2.1 Research on Thermal Imaging for medical diagnostics	9
	2.2 Deep Learning in Medical Image Processing	10
	2.3 Previous works on diabetic foot detection using deep learning and thermal imaging	11-14
3.	**CHAPTER 3: EXISTING SYSTEM**	**15-19**
	3.1 Traditional methods	15
	3.2 Limitations of current approaches	16
	3.3 Machine learning-based approaches Limitations	17

	3.4 Need for deep learning-based automated thermal image detection	18-19
4	**CHAPTER 4: MOBILENET-BASED PROPOSED SYSTEM**	**20-41**
	4.1 Why Mobile Net?	20
	4.2 Requirement Analysis	21-22
	4.3 Hardware Requirements	23
	4.4 Software Requirements	24
	4.5 Architecture	25
	4.6 Methodology	26-35
	4.7 System Design	36-41
5	**CHAPTER 5: RESULT ANALYSIS**	**42-56**
	5.1 Implementation	42-52
	5.2 Output	53-56
6	**CHAPTER 6: CONCLUSION AND FUTURE WORK**	**57-61**
	6.1 Conclusion	57
	6.2 Future work	58-61
7	**REFERENCES**	62

List of figures

S.NO	FIGURE NAME	PG.NO
1.	EVALUATION METRICS	11
2.	COMPARATIVE OVERVIEW OF APPROACHES	14
3.	ARCHITECTURE OF MOBILENETV2	25
4.	FLOW CHART OF METHODOLOGY	26
5.	FOLDER STRUCTURE	27
6.	PROJECT STRUCTURE	32
7.	DATASET FOLDERS	42
8.	HOME PAGE	53
9.	UPLOAD PAGE	53
10.	ANALYSIS PAGE	54
11.	REPORT PAGE	55

CHAPTER 1

INTRODUCTION

Diabetes mellitus is a prevalent global health issue, affecting millions of individuals and contributing to severe complications such as diabetic foot ulcers (DFU). If left undiagnosed or untreated, these ulcers can lead to infections, amputations, and increased mortality rates, significantly straining healthcare systems. Traditional diagnostic approaches for diabetes rely on blood tests, physical examinations, and Doppler ultrasound imaging. However, these methods can be time-consuming, invasive, and prone to human error, necessitating the development of an advanced, automated, and non-invasive detection system.

Recent advancements in **thermal imaging** and **deep learning** have paved the way for improved diabetic diagnosis through non-contact and AI-driven methods. **Convolutional Neural Networks (CNNs)** have demonstrated remarkable success in medical image classification by identifying patterns and temperature anomalies that might not be easily visible to the human eye. This project leverages Mobile Net, a lightweight and efficient deep learning architecture, to analyze thermal foot images and detect diabetes-related abnormalities based on temperature variations.

The motivation for this project arises from the limitations of conventional diagnostic approaches. Existing methods often require direct patient contact, increasing the risk of infections and diagnostic inconsistencies among healthcare practitioners. Additionally, subjective assessments can lead to variability in diagnoses, delaying necessary interventions. The primary objective of this project is to develop a fast, reliable, and scalable deep learning-based diagnostic tool for diabetes detection using thermal imaging. By automating the process of analyzing plantar thermo-grams, the system aims to provide a non-invasive screening method that improves early detection, reduces complications, and supports better clinical decision-making. Through rigorous experimentation and performance evaluation, this research seeks to demonstrate the real-world feasibility of AI-driven thermal diagnostics, ultimately contributing to improved healthcare outcomes and reducing diabetes-related complications **[1].**

1.1 Motivation:

The need for an automated diabetes detection system using thermal imaging and deep learning arises from the growing prevalence of diabetes and its severe complications, including. If undiagnosed or untreated, diabetes can lead to **peripheral neuropathy**, reducing sensation in the feet and increasing the risk of ulcers, infections, and lower-limb amputations. Traditional diagnostic methods—such as blood glucose testing, Doppler ultrasound, and clinical foot examinations—are often invasive, time-consuming, subjective, and dependent on expert interpretation**.** These limitations create an urgent need for a fast, automated, and non-invasive approach for early diabetes detection. Recent advancements in thermal imaging and deep learning**,** particularly Mobile Net-based Convolutional Neural Networks (CNNs)**,** have enabled more precise and scalable diagnostic solutions. By analysing temperature variations in foot thermo-grams**,** deep learning models can identify abnormalities associated with diabetes-induced vascular and neurological changes. This AI-driven system offers rapid, accurate, and real-time analysis**,** ensuring early detection and proactive intervention before complications arise. Let us delve into the key motivations:

i. Early Detection to Reduce Diabetes-Related Complications: Diabetes often progresses silently, with many patients remaining undiagnosed until serious complications develop. Temperature asymmetries in plantar thermo-grams can serve as early indicators of diabetes-induced circulatory dysfunction. This system aims to detect diabetes before visible symptoms appear**,** allowing for timely medical intervention and reducing the risk of severe complications, including diabetic foot ulcers and amputations.

ii. Overcoming the Limitations of Traditional Diagnostic Methods: Conventional diabetes detection relies on blood glucose monitoring, HbA1c tests, and physical assessments**,** which require direct patient contact, laboratory facilities, and skilled professionals. These methods can be costly, inaccessible in remote areas, and prone to human error. AI-powered thermal imaging analysis provides a non-contact, objective, and fully automated diagnostic approach, improving accuracy and accessibility.

iii. Leveraging Advances in Deep Learning for Medical Imaging: With recent breakthroughs in medical image analysis and deep learning, CNN-based models have demonstrated exceptional performance in feature extraction, pattern recognition, and classification of medical images. Mobile Net, a lightweight yet powerful deep learning architecture, is particularly suited for real-time analysis on edge devices such as smartphones and embedded systems, making diabetes detection more accessible and scalable.

iv. Improving Patient Outcomes and Quality of Life: Diabetes and its complications can significantly impact a patient's mobility, independence, and mental health. The fear of amputation, prolonged wound healing, and chronic infections contribute to psychological distress**.** By enabling early diagnosis through AI-driven thermal imaging**,** this system enhances preventive care strategies, reduces hospitalization rates, and ultimately improves patients' overall well-being**.**

v. Enhancing Accessibility through Remote Monitoring and Medical Health Solutions

Many diabetic individuals, particularly in rural and underserved regions**,** lack access to regular screenings and specialized healthcare facilities. This project bridges the gap by providing a low-cost, mobile-compatible AI-based system that can be integrated into telemedicine and mobile health (medical Health) platforms. Remote monitoring enables continuous assessment of foot health, reducing hospital visits while allowing doctors to track disease progression remotely**.**

In summary, the motivation behind this deep learning-based thermal imaging system for diabetic foot ulcer detection is driven by the urgent need to enhance early diagnosis, improve patient outcomes, and optimize healthcare resources. By leveraging artificial intelligence and image processing, this research aims to provide a reliable, non-invasive, and efficient solution that can be deployed in clinical settings and remote healthcare applications. Ultimately, the integration of AI-driven detection technologies reinforces early intervention strategies, reduces amputation risks, and enhances the overall quality of life for diabetic patients **[2].**

1.2 Problem Statement

The problem statement for the diabetes detection project using thermal foot imaging and Mobile Net-based deep learning revolves around the inadequacy of traditional methods in effectively identifying diabetic foot ulcers (DFUs) and related complications, particularly in

the face of increasing diabetes prevalence and the limitations of manual diagnostic techniques. Traditional DFU detection methods, such as physical examinations, Doppler ultrasound, and patient-reported symptoms, are often subjective, time-consuming, and prone to human error. These methods rely on visual inspection and predefined diagnostic criteria, making them inconsistent and less reliable for early detection.

The problem statement, therefore, centres on the need to develop a robust, reliable, and non-invasive diabetic foot ulcer detection system using thermal imaging and deep learning techniques. This system should be capable of accurately detecting temperature abnormalities, classifying healthy and affected foot regions, and providing real-time diagnostic insights with high precision and efficiency. Moreover, it should be scalable for widespread clinical use, adaptable to different patient conditions, and integrated into telemedicine platforms for remote monitoring and early intervention.

In Summary, The problem statement underscores the imperative to overcome the limitations of traditional diabetes detection methods through the development of an AI-driven, non-invasive screening solution using thermal imaging and deep learning. This project aims to enhance early detection, reduce the risk of complications, and improve patient outcomes, ultimately contributing to better healthcare accessibility and lower amputation rates in diabetic patients **[3].**

1.3 Objectives of the Project:

The primary objective of the diabetic detection project using deep learning and thermal imaging is to design, implement, and evaluate an automated system for early identification of diabetic foots. This involves utilizing deep learning algorithms, particularly Convolutional Neural Networks (CNNs), and thermal imaging techniques to create an accurate and efficient detection mechanism.

i. Designing a CNN-Based Model

The project aims to develop a Mobile Net-based deep learning model specifically tailored for diabetic foot detection. The model will be trained to analyse thermal foot images and identify temperature variations that indicate the presence of ulcers.

ii. Training on a Comprehensive Dataset

To ensure the model's accuracy and generalization capability, the project involves training it on a large dataset of thermal foot images, including both healthy and ulcer-affected feet. The dataset will be pre-processed and augmented to improve performance.

iii. Optimizing Model Performance

The project seeks to enhance the CNN-based model's performance through hyper parameter tuning, regularization techniques, and data augmentation. These optimizations aim to improve the model's accuracy, efficiency, and robustness in diabetes detection tasks.

iv. Evaluating Detection Accuracy

The system will be evaluated using various performance metrics, including accuracy, precision, recall, F1-score, and AUC-ROC curves, to ensure its reliability in detecting early-stage diabetic foot ulcers.

v. Assessing Robustness and Real-World Applicability

Beyond accuracy, the project aims to assess the real-world usability of the system by testing its performance under different conditions, such as varied lighting, skin tones, and foot positions. This ensures scalability for use in hospitals, clinics, and remote healthcare settings.

vi. Deployment and Integration

Finally, the project seeks to deploy and integrate the developed diabetic foot ulcer detection system into clinical applications and telemedicine platforms. The system should be user-friendly, scalable, and compatible with existing medical imaging tools to assist doctors and patients in monitoring foot health.

The diabetic foot ulcer detection project using deep learning and thermal imaging aims to develop a non-invasive, AI-powered screening tool that enables early detection and intervention, ultimately reducing amputation risks and improving diabetic patient care.

1.4 Scope

This project's scope encompasses the development of a deep learning-based diabetes detection system using thermal imaging for early diagnosis and monitoring. The system aims to analyse thermal foot images, identifying temperature variations that indicate potential ulcers. Additionally, the project includes optimizing the deep learning model for accuracy, efficiency, and real-world applicability, ensuring its usability in various healthcare settings.

The scope for diabetic foot detection using thermal imaging and AI is vast and continually expanding, driven by both technological advancements and the growing need for non-invasive diagnostic tools. As thermal imaging and deep learning technologies evolve, their potential applications in medical diagnostics become increasingly sophisticated, offering more precise and accessible screening methods compared to traditional manual foot inspections.

This AI-powered diagnostic tool can be integrated into various healthcare platforms, from hospital-based screening systems to portable mobile health applications that allow patients and caregivers to monitor foot health remotely. The broad applicability ensures that the technology benefits diabetic patients, healthcare professionals, telemedicine services, and preventive care programs, enabling early intervention and reducing the risk of severe complications.

Moreover, the combination of artificial intelligence and medical imaging has opened new possibilities for automated, data-driven diagnosis, allowing the system to continuously improve and adapt to different foot conditions and patient demographics. This capability enhances detection accuracy, enabling personalized healthcare solutions and supporting clinical decision-making **[4].**

1.5 Project Overview

The project aims to address the growing challenge of early diabetes detection by leveraging advancements in thermal imaging and deep learning technologies. By developing a non-invasive, AI-powered detection system, we seek to enhance early diagnosis, reduce amputation risks, and improve patient outcomes. Through extensive research and experimentation, this project contributes to modernizing diabetic foot care and supporting automated medical diagnostics.

The increasing prevalence of diabetic foot ulcers among diabetes patients worldwide has created an urgent need for efficient and reliable screening methods to prevent severe complications. This project focuses on utilizing thermal imaging and deep learning to develop an intelligent diabetes detection system that can accurately differentiate between healthy and ulcer-affected foot regions. By integrating Convolutional Neural Networks (CNNs) such as Mobile NetV2, this system aims to provide a scalable, cost-effective, and accessible solution that can be implemented in hospitals, clinics, and telemedicine platforms.

At the core of this project is the application of advanced image analysis techniques, including temperature mapping, feature extraction, and anomaly detection. High-resolution thermal foot images are captured under controlled conditions, and the system pre-processes these images to enhance contrast, detect thermal variations, and reduce noise. The deep learning model is then trained to identify key thermal patterns that indicate the presence of early-stage ulcers, enabling faster and more precise diagnosis.

CHAPTER 2

LITERATURE SURVEY

The field of diabetic diagnosis has witnessed a paradigm shift with the convergence of biomedical imaging technologies and intelligent computational systems, particularly artificial intelligence (AI) and deep learning (DL). The exponential rise in the prevalence of diabetes mellitus (DM)—a chronic metabolic disorder characterized by hyper glycemia has compelled the global medical community to explore novel diagnostic modalities that are faster, non-invasive, and more accurate than conventional clinical approaches.

According to the International Diabetes Federation (IDF), over 537 million people worldwide are living with diabetes as of 2021, a figure projected to surge to 643 million by 2030 and 783 million by 2045. One of the most serious consequences of diabetes is the development of diabetic foot complications, including diabetic foot ulcers (DFUs), neuropathy, peripheral vascular disease, and amputations. The early identification and prevention of such complications is critical to avoid severe morbidity and mortality.

Traditional diagnostic methods—comprising clinical observation, manual inspection of foot deformities, monofilament tests for neuropathy, and blood tests—often fail to detect the underlying pathology at an early stage. The inherent subjectivity in visual examination, inter-observer variability, delayed manifestation of symptoms, and patient discomfort in invasive procedures all contribute to diagnostic inefficiencies. These challenges necessitate the evolution of advanced diagnostic tools, especially those that can operate autonomously and accurately in early stages **[5].**

Key Literature Themes and Research Directions:

1. **Thermo-grams for Pre-Ulcer Detection:**
   Thermo-grams visualize heat distribution on the surface of the foot and can detect asymmetries or anomalies associated with inflammation and neuropathy. These early indicators can significantly reduce the incidence of DFUs if detected timely.

   As per Vardasca et al., temperature variations between corresponding regions of both feet (Angiosome-based analysis) offer high sensitivity for early diabetic foot detection.

2. **Feature Extraction Using Classical Image Processing Techniques:**
   Early studies relied on handcrafted features such as:
   - Grey Level Co-occurrence Matrix (GLCM) for texture analysis.
   - Discrete Wavelet Transform (DWT) for frequency domain feature extraction.
   - Local Binary Patterns (LBP) and Hu's Invariant Moments for shape and texture descriptors.

     These features, although effective to an extent, required extensive domain expertise and lacked generalizability across diverse datasets.
3. **Transfer Learning as a Game Changer in Medical Diagnostics:**
   Transfer learning leverages pre-trained CNN models (e.g., ResNet50, VGG16, Mobile Net) trained on large-scale datasets like Image Net and fine-tunes them on smaller domain-specific datasets such as thermo-grams. It reduces training time, mitigates the problem of insufficient data, and improves model convergence.
   a. **Cruz-Vega et al.** used **DFT Net**, a novel CNN architecture inspired by Alex Net, and compared it with traditional ML models. Their work highlighted the superiority of deep learning in feature discovery and classification accuracy.

2.1 Research on Thermal Imaging for Medical Diagnostics:

Thermal imaging, also known as infrared thermography (IRT), is a rapidly emerging technique in the domain of medical diagnostics, offering a non-invasive, contactless, and radiation-free approach to visualizing temperature distribution across human body surfaces. Its use in the early detection of diabetic foot complications is gaining momentum, primarily due to its ability to identify subtle physiological changes before clinical symptoms visibly manifest.

I. **Deep Learning Empowered Thermal Imaging:**

With advancements in AI, particularly in deep learning, the diagnostic potential of thermal imaging has drastically improved. Instead of relying solely on handcrafted features, Convolutional Neural Networks (CNNs) can now directly learn discriminative thermal patterns from raw thermo-grams.

I. **Deep Learning Benefits:**
   - End-to-end learning pipelines – no manual feature engineering required.
   - High sensitivity and specificity, even on small temperature differentials.
   - Real-time diagnosis during mass screening.
   - Robustness to noise and environmental variations.

II. **Notable Use-Cases from Literature:**
   - **Cruz-Vega et al.** used DFT Net to classify thermo-grams and showed how CNNs outperformed traditional methods.
   - **Anaya-Isaza et al.** applied data augmentation and advanced pre-processing, improving DL model generalization.
   - **Nithya Rajagopalan et al.** used Mobile Netv2 on a diabetic foot dataset and achieved 93% accuracy, highlighting the viability of thermal + deep learning integration.

2.2 Deep Learning in Medical Image Processing:

The evolution of medical image analysis has undergone a significant revolution with the advent of Deep Learning (DL) a subset of Artificial Intelligence (AI) that mimics the human brain's functioning through artificial neural networks. Among its various domains, Convolutional Neural Networks (CNNs) have had a profound impact on medical image processing, enabling automated, accurate, and scalable diagnostic systems.

Traditional medical imaging systems heavily depended on the domain knowledge of medical experts, manual inspection, and handcrafted feature extraction techniques for disease diagnosis. However, such methods are limited by human subjectivity, inter-observer variability, time consumption, and diagnostic inaccuracies, especially in complex pathological cases. Deep Learning eliminates these limitations by providing an end-to-end solution where the model itself learns the most important features from raw image data without explicit programming or manual engineering.

2.2.1 Why Deep Learning in Medical Imaging?

Medical images be it X-rays, MRIs, CT scans, or thermo-grams are often high-dimensional, complex, and laden with subtle features that may be hard to interpret even by trained clinicians. Deep Learning provides a framework to extract multi-level hierarchical features, which can distinguish between healthy and pathological tissue with remarkable accuracy.

Key strengths of DL in medical imaging include:

- High classification accuracy
- Automated feature extraction
- Adaptability to various imaging modalities
- Transfer learning from pre-trained models

Metric	Description
Accuracy	Correct predictions over total predictions
Precision	True Positives / (True Positives + False Positives)
Recall (Sensitivity)	True Positives / (True Positives + False Negatives)
F1 Score	Harmonic mean of Precision and Recall
AUC-ROC	Area under ROC curve indicating classification performance

Fig 1: Evaluation Metrics in DL-based Medical Imaging

2.3 Clinical Relevance of Diabetic Foot Detection:

DFUs often develop due to peripheral neuropathy, ischemia, and improper mechanical loading of the foot. These factors lead to localized areas of increased or decreased temperature subtle anomalies which, although invisible to the naked eye, are clearly captured in thermo-grams. Manual detection and interpretation of these anomalies are both time-consuming and prone to errors, making AI-enabled thermal image analysis a game-changer in diabetic foot care.

2.3.1 Review of Major Contributions in Literature:

Let's now explore some of the key contributions and methodologies adopted by researchers across various stages of diabetic foot detection and classification using thermal imaging:

2.3.1.1 Adam et al. – Hybrid Feature Extraction with SVM Classifier:

Adam et al. used **Discrete Wavelet Transform (DWT)**, **Grey Level Co-occurrence Matrix (GLCM)**, **Local Ternary Pattern (LTP)**, and **Higher Order Statistics (HOS)** for feature extraction. These handcrafted features captured statistical, spatial, and frequency domain information from thermo-grams **[6].**

- **Classifier Used:** Support Vector Machine (SVM)
- **Accuracy Achieved:** ~89%

- **Limitations:** Required complex pre-processing and lacked adaptability to unseen variations in thermal data.

2.3.1.2 Prabhu et al. – Unsupervised Clustering and Segmentation:

Prabhu et al. proposed a method for **automatic plantar foot segmentation using C-means clustering**, which separated distinct thermal zones based on pixel intensity. Post-segmentation, classification was performed using a conventional ML model **[7].**

- **Strength:** Improved localization of diabetic hotspots.
- **Limitation:** Semi-automated and required manual intervention for cluster tuning.

2.3.1.3 Josephine Selle et al. – Texture-Based GLCM Feature Analysis:

This study focused on extracting **15 GLCM features** such as entropy, contrast, homogeneity, and energy. These features were fed into an SVM for binary classification (diabetic vs. non-diabetic foot) **[8].**

- **Accuracy Achieved:** 96.42%
- **Merit:** High classification accuracy using handcrafted texture features.
- **Limitation:** Heavily reliant on optimal feature selection.

2.3.1.4 Nag et al. – Hybrid Statistical Approach with Decision Trees:

Nag et al. applied a hybrid approach using **first-order statistics**, **GLCM**, and **Wavelet-based features** with a **Decision Tree (DT)** classifier **[9].**

- **Accuracy Achieved:** 94.75%
- **Advantage:** Simple architecture with interpretable features.
- **Limitation:** Not robust for generalization on new datasets.

2.3.1.5 Cruz-Vega et al. – Deep Learning with DFT Net:

One of the landmark contributions in diabetic foot thermo-gram analysis, Cruz-Vega et al. proposed **DFT Net**, a custom CNN architecture inspired by Alex Net but optimized for thermal imaging datasets **[10].**

- **Comparison Made:** DFT Net vs. SVM vs. MLP vs. traditional CNN.
- **Accuracy:** Over 95% classification accuracy.

- **Highlights:**
  - Demonstrated the superiority of deep learning over classical ML.
  - Achieved significant performance without extensive manual pre-processing.
  - Leveraged **data augmentation** and **adaptive training parameters**.

2.3.1.6 Anaya-Isaza et al. – Data Augmentation and Transfer Learning:

Anaya-Isaza and Zequera-Diaz emphasized the importance of **advanced data augmentation strategies** to mitigate the challenges of small thermal datasets. They utilized techniques such as:

- PCA
- ICA
- NMF
- LDA
- Kernel PCA
- Dictionary Learning

They trained multiple **pre-trained models** like VGG16 and ResNet50v2 through transfer learning.

- **Key Achievement:** Boosted model performance beyond conventional transformations like flipping and rotation.
- **Contribution:** Provided a robust framework for improving generalizability of CNNs in medical imaging.

2.3.1.7 Khandakar et al. – Hybrid ML + DL Model:

This study used a hybrid pipeline combining traditional ML and deep learning:

- Feature Extraction: Thermal symmetry features and image-derived metrics.
- Models: CNN + SVM + Random Forest.
- **Outcome:** Enhanced prediction accuracy (~95%) and better model interpretability.

Study	Feature Type	Model	Accuracy	Notable Strength
Adam et al.	GLCM + DWT + HOS	SVM	~89%	Statistical feature set
Prabhu et al.	Unsupervised Segmentation	ML Classifier	Moderate	Enhanced localization
Selle et al.	GLCM	SVM	92.42%	Texture-based analysis
Nag et al.	GLCM + Wavelet	Decision Tree	95.78%	Hybrid statistical model
Cruz-Vega et al.	Raw Image	DFTNet	>94%	Custom CNN, high performance
Anaya-Isaza et al.	Raw Image + Augmentation	ResNet50, VGG16	~92-94%	Transfer learning
Khandakar et al.	Hybrid	CNN + ML	~96%	Balanced approach

Fig 2: Comparative Overview of Approaches

CHAPTER 3

EXISTING SYSTEM

3.1 Traditional Methods (Clinical Observation, Blood Tests, Manual Analysis of Foot Images):

The conventional diagnostic techniques for diabetes and diabetic foot complications have long been the cornerstone of clinical assessment. However, these traditional methods, though widely used, often fall short in early detection and prevention of severe conditions like Diabetic Foot Ulcers (DFUs), neuropathy, and ischemia **[11].**

- **Common Traditional Methods Include:**

1. **Clinical Observation and Physical Examination:**
   - Visual inspection of skin texture, color, dryness, cracks, calluses, and deformities (e.g., Charcot foot, hammer toes).
   - Examination for signs of inflammation, infection, or pressure-induced lesions.
2. **Neuropathy Assessment Tests:**
   - Monofilament Test: 10g Semmes-Weinstein monofilament is used to test for tactile sensation loss.
   - Vibration Test: 128 Hz tuning fork checks for vibratory perception.
   - Pinprick and Temperature Sensation Tests.
   - Ankle Reflex Testing for motor nerve damage.
3. **Blood Circulation Evaluation:**
   - Palpation of Dorsalis Pedis and Posterior Tibial Pulse.
   - Capillary Refill Time (CRT) assessment.
   - Ankle Brachial Index (ABI): Ratio of ankle to arm blood pressure for evaluating peripheral artery disease (PAD).
   - Doppler Ultrasound: Imaging of blood flow in arteries and veins.
4. **Blood Tests and Laboratory Evaluations:**

- Fasting Blood Glucose and HbA1c Tests to evaluate glycemic control.
- Serum lipid profile, creatinine, CRP levels for comorbidity screening.

5. **Manual Analysis of Foot Images:**
    - Clinical photographs of foot ulcers or deformities are reviewed periodically.
    - Often used in retrospective analysis or documentation.

➢ **Drawbacks of Traditional Methods:**

- Subjective interpretation and operator dependency.
- Late-stage detection (often after ulcer formation).
- Invasive tests may cause discomfort.
- Not scalable for mass screening.
- Limited to visible symptoms—cannot detect internal thermal anomalies.

While these methods are effective in clinical settings, they are reactive, not proactive focusing on visible symptoms rather than detecting underlying physiological changes early.

3.2 Limitations of Current Approaches:

Despite their wide use, the traditional diagnostic methods have multiple critical limitations that make them inadequate for early detection and prevention-focused diabetic foot care.

Key Limitations:

1. **Lack of Early Detection Capability:**
    - Most clinical signs (ulcers, calluses, infections) appear only in advanced stages of neuropathy or ischemia.
    - Early micro-vascular deterioration or thermal imbalance remains undetected.
2. **High Inter-Observer Variability:**
    - Diagnostic outcome depends on physician expertise, experience, and subjective judgment.
    - Different clinicians may interpret symptoms differently.
3. **Time-Consuming and Resource-Intensive:**
    - Manual tests like monofilament or tuning fork assessments are time-consuming for each patient.

- Repeated hospital visits are required for follow-ups.

4. **Invasive or Semi-Invasive Nature:**
   - Some procedures cause patient discomfort (e.g., ABI, Doppler scans).
   - Risks of infection or misinterpretation exist.
5. **Limited Scope in Mass Screening:**
   - Traditional methods are not suitable for large-scale diabetic screening camps or public health programs due to logistical challenges.
6. **Inadequate Predictive Analytics:**
   - Conventional methods are diagnostic, not predictive.
   - No foresight into future ulcer development or progression trajectory.
7. **Cost Implications:**
   - Routine tests and hospital visits increase economic burden on patients, especially in low-resource settings.

❖ **Clinical Insight:**

Due to these limitations, there's a rising need for more automated, scalable, non-invasive, and early detection tools that can supplement or even replace traditional methods.

3.3 Machine Learning-Based Approaches (Limitations of Simple ML Models in Medical Imaging)

With the advent of data-driven healthcare, researchers began exploring machine learning (ML) algorithms to automate diagnosis using thermal and visual images. ML models were among the first technological upgrades to conventional diagnostics.

➢ Common ML Techniques Used:
   - Support Vector Machines (SVM)
   - Decision Trees (DT)
   - Random Forest (RF)
   - k-Nearest Neighbours (k-NN)
   - Naive Bayes Classifier

- Input Feature Engineering:

  Unlike DL, ML requires manual design of features extracted from images:

  - Texture Features: GLCM, LBP, LTE
  - Statistical Metrics: Entropy, Mean Intensity, Skewness, Kurtosis
  - Wavelet Features: DWT coefficients
  - Shape Descriptors: Hu's invariant moments

These features are computed from thermo-grams or grayscale foot images and used to train ML classifiers to distinguish between healthy and diabetic foot images **[12].**

- **Limitations of ML Models in Medical Imaging:**

1. **Manual Feature Engineering:**
   - Requires domain knowledge and trial-and-error testing to select optimal features.
   - Misses complex or abstract patterns in thermal images.
2. **Curse of Dimensionality:**
   - Large number of features may lead to over fitting, reducing robustness.
3. **Scalability Issues:**
   - ML models struggle to scale in real-time clinical settings with large image volumes.
4. **No Hierarchical Feature Learning:**
   - ML algorithms can't learn multi-level features (e.g., edge → shape → object).
   - Hence, lower accuracy compared to CNNs in high-resolution thermal analysis.
5. **Dependency on Pre-processing Pipelines:**
   - ML models require extensive pre-processing, segmentation, and noise reduction before classification.

3.4 Need for Deep Learning-Based Automated Thermal Image Detection:

The increasing complexity and volume of diagnostic data, coupled with the limitations of traditional and ML-based methods, have created an urgent demand for **deep learning-based, automated, and intelligent diagnostic solutions**.

- **Why Deep Learning is the Game Changer:**
  - Learns hierarchical features directly from images (low-level to high-level).
  - Requires minimal human intervention.
  - Supports real-time inference and diagnosis.
  - Combines image classification, localization, and interpretation into a single unified pipeline.

DEEP LEARNING + THERMAL IMAGING = POWERFUL DIAGNOSTIC TOOL

CNNs can detect thermal asymmetries, vascular occlusions, skin temperature gradients, and subtle deformities more efficiently than any human observer or ML model.

- Transfer learning allows leveraging pre-trained models like ResNet50, VGG16, and Mobile Net to build high-accuracy systems even with limited thermo-graphic datasets.
- Techniques like Grad-CAM and saliency maps bring explainability into deep learning, making them transparent and trustworthy for clinical use.

- **Clinical Advantages:**
  - Early prediction of ulcer-prone regions before tissue breakdown.
  - Reduced dependency on invasive tests.
  - Faster screening in mass diagnosis camps or rural outreach programs.
  - Lower burden on healthcare systems.
  - Seamless integration with telemedicine platforms.

CHAPTER 4

MOBILENET BASED PROPOSED SYSTEM

The proposed system aims to detect diabetes using thermal imaging and deep learning techniques. By analysing input foot thermal images and comparing them with a dataset of labelled diabetic and non-diabetic images, the system utilizes the **Mobile Net** model and image processing algorithms to determine whether a person is diabetic or non-diabetic.

4.1 WHY MOBILE NET ?

Deep learning-based diabetes detection systems leveraging Mobile Net and thermal imaging offer several advantages over traditional diagnostic methods. These advanced technologies enable more accurate, efficient, and reliable detection, contributing to improved early diagnosis and patient care. Below are some key advantages of using Mobile Net in diabetes detection:

1. Enhanced Accuracy and Reliability: Mobile Net, a convolutional neural network (CNN), is optimized for learning complex patterns from medical imaging data. By training on a large dataset of diabetic and non-diabetic foot thermal images, Mobile Net can recognize subtle temperature variations that may not be visible to the human eye. As a result, the system achieves high accuracy in classification while reducing false positives and false negatives, ensuring more reliable diagnoses.

2. Efficient and Lightweight Model: Unlike traditional deep learning models such as VGG16 or Res Net, Mobile Net is optimized for speed and efficiency. It uses depth-wise separable convolutions to reduce computational complexity, making it suitable for real-time applications and deployment on edge devices or mobile healthcare platforms. This efficiency enables faster and more accessible diabetes screening, even in low-resource settings.

3. Adaptability to Different Patient Profiles: One of Mobile Net's significant advantages is its adaptability to diverse patient populations and varying thermal image conditions. The model can be fine-tuned with new datasets to improve its accuracy across different demographics, ensuring broad applicability and robustness against variations in skin tone, age, and temperature distribution.

4. **Real-time Detection and Instant Feedback:** Mobile Net's optimized architecture allows for real-time thermal image analysis, providing instant feedback to healthcare professionals

and patients. This feature is crucial in clinical and telemedicine applications, where quick and reliable results can aid in early diagnosis and timely intervention. Faster processing also improves patient experience by reducing wait times for diagnostic results.

5. Scalability and Cost-effectiveness: Mobile Net-based diabetes detection systems offer scalability and cost-effectiveness, particularly for large-scale screening programs. Once trained, a single model can process thousands of thermal images with minimal computational resources, making it ideal for deployment in hospitals, clinics, and community health programs. Additionally, automated diagnosis reduces the need for expensive laboratory tests and manual screening, leading to cost savings and increased efficiency in healthcare services.

6. Continuous Learning and Model Improvement: Deep learning models, including Mobile Net, have the advantage of continuous learning through retraining with new data. By incorporating additional thermal images and patient feedback, the model can adapt to emerging trends in diabetes diagnosis and improve its performance over time. Regular updates ensure that the system remains accurate and up to date with the latest medical research and advancements.

In conclusion , diabetes detection systems leveraging Mobile Net and thermal imaging provide numerous advantages over traditional diagnostic methods. These include enhanced accuracy, efficiency, real-time detection, scalability, continuous learning, and seamless integration into healthcare workflows. By adopting deep learning-powered medical imaging solutions, healthcare providers can improve early diagnosis, enhance patient outcomes, and contribute to more accessible and cost-effective diabetes screening solutions **[13]**.

4.2 Requirement Analysis

Functional Requirements:

Functional requirements outline the specific capabilities and features that a diabetes detection system using deep learning and thermal imaging should possess to meet its intended objectives. Here are some functional requirements for such a system.

1. Thermal Image Acquisition and Pre-processing:

- The system should be capable of acquiring thermal foot images from various sources, such as thermal cameras or digital databases.

- It should pre-process acquired images to enhance quality, remove noise, and standardize image characteristics for analysis.

2. **Feature Extraction:**

- The system should extract relevant features from thermal images using advanced image processing techniques.
- Features should include temperature variations, asymmetry detection, and heat distribution across foot regions to determine diabetic and non-diabetic conditions.

3. **Model Training and Optimization:**

- The system should train deep learning models, such as Mobile Net, using labelled datasets of diabetic and non-diabetic foot images.
- It should employ optimization techniques to fine-tune model parameters and improve classification accuracy.

4. **Real-time Detection:**

- The system should be capable of real-time diabetes detection upon image upload.
- It should provide timely feedback on the diagnosis to users or medical professionals.

5. **User Interface:**

- The system should have a user-friendly interface for interaction, allowing users to upload thermal images and receive detection results.
- The interface should provide clear feedback on the diagnosis, including confidence scores or probability estimates.

6. **Scalability and Performance:**

- The system should scale efficiently to handle large volumes of thermal images and processing tasks.
- It should demonstrate high performance in terms of detection accuracy, processing speed, and resource utilization.

Non-Functional Requirements:

1. Adaptability:

- The system should adapt to new diabetes detection methodologies, with regular updates to models and algorithms.

2. Response Time:

- Ensure quick response times for image analysis and detection, meeting real-time processing requirements.

3. Scalability:

- The system should handle scalability effectively, accommodating an increasing number of users and processing a growing volume of thermal images without significant performance degradation.

4. Reliability:

- Ensure consistent and accurate detection results across different datasets and scenarios to maintain system reliability.

5. Security:

- Implement security measures to protect patient data and ensure the confidentiality of processed images.

6. Interpretability:

- Provide explanations for detection decisions to enhance the interpretability of the diabetes diagnosis process.

4.3 Hardware Requirements:

Processor: Intel Core i5 or higher

Hard Disk: Minimum 250GB

Keyboard: Standard Keyboard

Mouse: Standard Mouse

Monitor: Full HD (1920x1080) Display

RAM: 16GB or higher

4.4 Software Requirements:

Operating System: Windows 10 or higher

Server-side Script: Not applicable

Programming Language: Python

Libraries: OpenCV, NumPy, scikit-learn, TensorFlow, Flask, Django

IDE/Workbench: PyCharm, Jupyter Notebook

Technology: Deep Learning, Image Processing, MobileNet Model

Database: Not applicable

Deployment: Standalone application

Python:

Python is a high-level programming language known for its simplicity, readability, and versatility. It is widely used in various fields, including artificial intelligence, data science, and deep learning. Its extensive libraries and strong community support make it an ideal choice for implementing diabetes detection models.

Open CV:

Open CV is an open-source computer vision library that provides essential tools for image processing. It is used in this project for pre-processing thermal images, extracting relevant temperature patterns, and enhancing image quality before feeding it into the Mobile Net model.

Streamlit:

Streamlit is a lightweight and flexible web framework for Python, enabling the development of web-based applications. It is used in this project to create a simple interface where users can upload thermal images and receive real-time diabetes detection results **[14].**

Scikit-learn:

Scikit-learn is a machine learning library that provides tools for classification, regression, and model evaluation. It is used in this project for statistical analysis and performance evaluation of the trained Mobile Net model.

Tensor Flow:

Tensor Flow, developed by Google, is a widely used deep learning framework. It enables the implementation and training of Mobile Net for diabetes detection, allowing efficient model deployment and optimization for high accuracy. With Mobile Net's lightweight structure, the system can provide quick and accurate diagnoses without requiring heavy computational resources **[16].**

4.5 Architecture

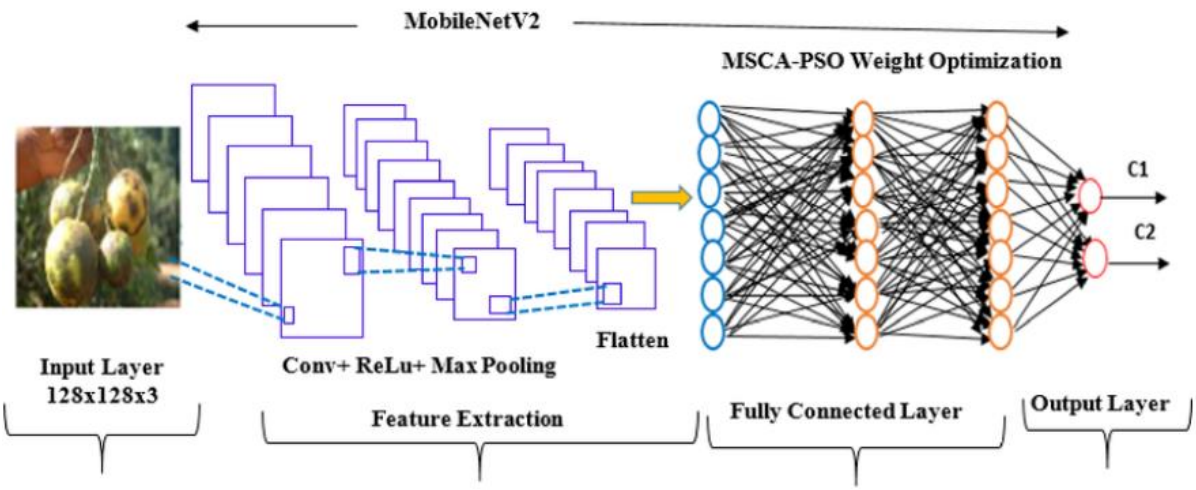


Fig 3:MOBILENETV2 ARCHITECTURE

4.6. Methodology

The Diabetic Foot Detection project aims to develop a web-based application that uses thermal images of feet to predict diabetic foot conditions, extract thermal features, visualize CNN feature maps, and provide exercise recommendations. The methodology is divided into several key phases: data preparation, model development, feature extraction, application development, and deployment. Below is a detailed breakdown of each phase.

Fig 4: Flowchart of Methodology

1. Data Preparation

The dataset consists of thermal images of feet, categorized into diabetic and non-diabetic groups. The data preparation phase involves organizing, pre-processing, and splitting the dataset for training, validation, and testing.

- **Data Collection**:

Thermal images of feet were collected from diabetic and non-diabetic individuals.

The dataset is stored in C:/Users/FYP/Data/raw_data/.

Images are labelled as "diabetic" or "non-diabetic" based on clinical diagnoses.

- **Data Organization**:

The dataset was organized into a structured directory:

```
├── train/
│   ├── diabetic/
│   │   └── (images, e.g., DM001_F_L.png)
│   └── nondiabetic/
│       └── (images, e.g., CG001_M_L.png)
├── val/
```

```
│   ├── diabetic/
│   └── nondiabetic/
├── test/
│   ├── diabetic/
│   │   └── (images, e.g., DM009_F_R.png)
│   └── nondiabetic/
```

FIG 5: FOLDER STRUCTURE

Images were split into training (70%), validation (15%), and testing (15%) sets to ensure robust model evaluation.

2. Data Pre-Processing:

Images were resized to a uniform size of 224x224 pixels to match the input requirements of the CNN model. Pixel values were normalized to the range [0, 1] by dividing by 255, as thermal images are typically grayscale or RGB but need to be scaled for model input. Data augmentation techniques (e.g., rotation, flipping) were applied during training to increase the dataset's diversity and prevent over fitting.

3. Model Development

A Convolutional Neural Network (CNN) was developed to classify thermal images as "Diabetic" or "Non-Diabetic." The model development phase includes architecture design, training, and evaluation.

CNN Architecture:

A CNN model was designed with multiple convolutional layers, followed by pooling layers, to extract spatial features from thermal images.

The architecture includes:

Input layer: Accepts 224x224x3 images (RGB or converted grayscale).

Convolutional layers: Extract features such as edges and textures.

Pooling layers: Reduce spatial dimensions while preserving important features.

Fully connected layers: Perform classification.

Output layer: A sigmoid activation function to output a probability score (0 to 1) for binary classification (Diabetic vs. Non-Diabetic).

A specific layer (block_3_expand) was used for feature extraction to visualize intermediate feature maps.

Model Training: The model was trained using the training dataset (Data/split_data/train/).

Binary cross-entropy loss was used as the loss function, suitable for binary classification.

The Adam optimizer was employed to optimize the model's weights.

Training was performed over multiple epochs, with early stopping based on validation loss to prevent over fitting.

The validation dataset (data/split data/val/) was used to monitor the model's performance during training.

Model Evaluation:

The trained model was evaluated on the test dataset (Data/split data/test/).

Metrics such as accuracy, precision, recall, and F1-score were computed to assess the model's performance.

The model was saved as model.keras in C:/Users/FYP/models/cnn_model/ for use in the application.

Feature Extraction Model:

- A separate model was created for feature extraction by extracting the output of the block_3_expand layer from the trained CNN.

This model is used to generate feature maps for visualization, helping to understand which regions of the image the CNN focuses on during classification.

4. Feature Extraction and Analysis

Thermal image features were extracted to provide additional insights into foot health, such as temperature distribution and texture properties. These features are used for both analysis and exercise recommendations.

Temperature Mapping:

Pixel values in the thermal images were scaled to a realistic temperature range (20°C to 40°C) using the formula:

```
temperatures = 20 + (img_array / 255.0) * (40 - 20)
```

- This mapping assumes that the pixel intensities in the thermal image correspond linearly to temperatures within this range.

Temperature Statistics:

- Computed statistical features from the temperature map:
- Mean temperature (mean_temp): Average temperature across the foot.
- Standard deviation (std_dev): Variation in temperature, indicating uniformity.
- Skewness (skewness): Asymmetry in the temperature distribution.
- Minimum temperature (min_temp): Lowest temperature in the image.
- Maximum temperature (max_temp): Highest temperature in the image.
- Regional temperatures were calculated for specific areas of the foot:
- Heel region (heel_temp): Lower-left region of the image.
- Toe region (toe_temp): Upper-middle region of the image.
- Arch region (arch_temp): Middle-right region of the image.

Texture Features (GLCM):

The Grey-Level Co-occurrence Matrix (GLCM) was computed using the skimage.feature.graycomatrix function to analyse the texture of the temperature map.

GLCM properties were extracted:

- Contrast (contrast): Measures local variations in temperature.
- Energy (energy): Measures uniformity of the temperature distribution.
- Homogeneity (homogeneity): Measures closeness of the temperature distribution to the GLCM diagonal.
- Correlation (correlation): Measures linear dependency of temperature values.**[15]**

Hotspots and Circulation Score:

Hotspots (hotspots): Identified as the number of pixels with temperatures above the 90th percentile, indicating potential areas of inflammation.

Circulation score (circulation score): Calculated as:

```
circulation_score = 1 - (std_dev / max_temp) - (hotspots / total_pixels)
```

A higher circulation score indicates better circulation (more uniform temperature distribution and fewer hotspots).

Normal Ranges:

Normal ranges for each feature were defined based on typical values for healthy feet:

- Contrast: 0–50
- Energy: 0.1–0.5
- Homogeneity: 0.5–1.0
- Correlation: 0.7–1.0
- Mean Temperature: 30–34°C
- Standard Deviation: 1–3°C
- Skewness: -0.5 to 0.5
- Min Temperature: 28–32°C
- Max Temperature: 32–36°C
- Heel Temperature: 30–34°C
- Toe Temperature: 28–32°C
- Arch Temperature: 30–34°C
- Hotspots: 0–5
- Circulation Score: 0.6–1.0

These ranges were used to classify features as "Normal" or "Abnormal" in the analysis.

5. Exercise Recommendation System

An exercise recommendation system was developed to provide personalized suggestions based on the circulation score and maximum temperature of the foot.

Recommendation Logic:

High Temperature (max_temp > 37°C):

Suggest rest and cooling therapies to reduce inflammation:

- Rest and Elevation: Elevate feet for 15–20 minutes.
- Cooling Therapy: Apply a cool compress for 10 minutes.
- Avoid Weight-Bearing Exercises: Refrain from walking or standing.

Poor Circulation (circulation_score < 0.3 or max_temp < 30°C):

Suggest gentle exercises to stimulate blood flow:

- Toe Wiggling: Wiggle toes for 5 minutes.
- Ankle Rotations: Rotate ankles for 5 minutes.
- Seated Leg Raises: Lift legs for 10 seconds, repeat 10 times.

Moderate Circulation (0.3 ≤ circulation_score < 0.6):

Suggest moderate exercises to improve circulation:

- Calf Raises: Rise onto toes, hold for 3 seconds, repeat 15 times.
- Walking: Walk for 10–15 minutes at a moderate pace.
- Foot Massage: Massage feet for 5–10 minutes.

Good Circulation (circulation_score ≥ 0.6):

Suggest active exercises to maintain circulation:

- Brisk Walking: Walk briskly for 20–30 minutes.
- Toe Taps: Tap toes for 1 minute, repeat 5 times.
- Stretching: Perform foot and calf stretches for 10 minutes.

Implementation:

The recommend_exercises function in the code implements this logic, returning a list of exercise recommendations based on the input circulation_score and max_temp.

6. Application Development (Streamlit App)

The Streamlit web application was developed to provide a user-friendly interface for uploading thermal images, viewing analysis results, and downloading a PDF report.

1. **Environment Setup**:

A Python environment was set up with the required dependencies listed in requirements.txt:

- streamlit: For the web interface.
- tensorflow: For model loading and prediction.
- numpy, pillow, scikit-image, scipy, matplotlib: For image processing and feature extraction.
- fpdf: For generating PDF reports.

The project structure was organized as:

```
C:/Users/harsh/FYP/
├── Data/
│   ├── split_data/
│   │   ├── train/
│   │   ├── val/
│   │   └── test/
│   └── raw_data/
├── models/
│   ├── cnn_model/
│   │   └── model.keras
├── src/
│   ├── streamlit_app/
│   │   ├── uploads/
│   │   └── app.py
│   └── predict_cnn.py
├── requirements.txt
```

```
└── README.md
```

FIG 6: PROJECT STRUCTURE

Streamlit App Implementation:

2. User Interface:

The app (app.py) includes input fields for the patient's name and age.

Two file uploaders allow users to upload thermal images of the left and right feet.

A "Predict" button triggers the analysis.

3. Prediction:

The predict_cnn function loads the image, preprocesses it, and uses the trained CNN model to predict whether the foot is "Diabetic" or "Non-Diabetic."

It also extracts feature maps from the block_3_expand layer for visualization.

4. Feature Extraction:

The extract_features function computes temperature statistics, GLCM texture features, regional temperatures, hotspots, and the circulation score.

Features are compared against normal ranges to classify them as "Normal" or "Abnormal."

5. Feature Map Visualization:

The plot_feature_maps function generates a visualization of the feature maps extracted from the CNN, showing six filters as a grid of images.

6. Exercise Recommendations:

The recommend_exercises function provides exercise suggestions based on the circulation score and maximum temperature.

7. Results Display:

The app displays the prediction, confidence, feature maps, thermal features (with normal ranges and status), and exercise recommendations for each foot.

Features outside the normal range are highlighted in red, while those within the range are highlighted in green.

8. PDF Report Generation:

The generate_pdf_report function creates a PDF report using the FPDF library.

The report includes the patient's name, age, prediction results, thermal features, and exercise recommendations.

En dashes in normal range strings were replaced with hyphens to avoid encoding issues with latin1.

9. Error Handling and Logging:

Logging was implemented to track the app's execution, including model loading, file uploads, predictions, and errors.

Error handling ensures the app provides meaningful feedback for issues like missing files, invalid inputs, or prediction failures.

7. Testing and Validation

The application was tested to ensure functionality and reliability.

Unit Testing:

- The predict_cnn function was tested standalone (predict_cnn.py) using a sample image (DM009_F_R.png) to verify that predictions are generated correctly.
- The extract_features function was tested to ensure accurate computation of thermal features.

Integration Testing:

The Streamlit app was tested by uploading sample images (CG003_M_L.png and CG003_M_R.png) and verifying that:

- Predictions are accurate and match expected outcomes.
- Feature maps are displayed correctly.
- Thermal features are computed and compared against normal ranges.
- Exercise recommendations are appropriate based on the circulation score and maximum temperature.
- The PDF report is generated and downloaded without errors.

Error Handling Testing:

Tested edge cases, such as uploading no images, invalid images, or missing patient details, to ensure the app provides appropriate error messages.

Fixed issues like the UnicodeEncodeError in PDF generation by replacing en dashes with hyphens.

8. Deployment

The application was prepared for deployment to make it accessible online.

1. **Local Deployment**:
   - The app was run locally using:

```
cd C:/Users/harsh/FYP/src/streamlit_app

streamlit run app.py
```

   - It was accessible at http://localhost:8501.
2. **Online Deployment**:

   The app can be deployed on Streamlit Community Cloud:

- Push the project to a GitHub repository.
- Sign up at streamlit.io and create a new app.
- Link the GitHub repository and specify the app file (src/streamlit_app/app.py).
- Deploy the app and access it via the provided URL.

This methodology ensures a robust, user-friendly, and clinically relevant tool for diabetic foot detection, with potential for future enhancements to improve accuracy and usability.

By implementing MobileNetV2**,** this system efficiently classifies diabetic and non-diabetic individuals based on thermal imaging. The combination of deep learning, feature extraction, and statistical analysis ensures high accuracy, making it a reliable tool for early diabetes detection and medical diagnostics**.** The methodology ensures scalability, real-time diagnosis, and integration into clinical applications, benefiting both patients and healthcare professionals**.**

4.7. System Design

The system design for diabetic foot detection using thermal imaging and a custom CNN-based deep learning model consists of several key components, including hardware for image acquisition, pre-processing and feature extraction modules, a deep learning model for classification, a user interface for interaction and reporting, and considerations for integration, scalability, security, and maintenance. Below is a detailed breakdown of the system design:

1.Hardware Requirements

Thermal Camera: A high-resolution thermal camera capable of capturing temperature variations in the plantar region of the foot. This is critical for detecting thermal patterns that may indicate diabetic foot complications, such as hotspots or temperature asymmetry.

Computer System: A system equipped with a high-performance CPU and, optionally, a GPU to handle the computational requirements of deep learning model inference, image processing, and feature extraction. For example, a system with an Intel i7 processor and an NVIDIA GPU (e.g., GTX 1650 or higher) is suitable for efficient processing.

Storage: Sufficient storage capacity to manage the thermal image dataset, extracted features, trained models, and temporary files (e.g., uploaded images and generated reports). A minimum of 500 GB of SSD storage is recommended to handle the dataset and application files.

2. Software and Frameworks

Image Processing Libraries:

Pillow (PIL): Used for loading and pre-processing thermal images, such as resizing and converting images to arrays.

scikit-image: Utilized for advanced image processing tasks, including computing the Gray-Level Co-occurrence Matrix (GLCM) for texture feature extraction.

Matplotlib: Employed for visualizing feature maps extracted from the CNN model.

Deep Learning Framework:

TensorFlow and Keras: Used for loading the pre-trained custom CNN model, performing predictions, and extracting feature maps from intermediate layers (e.g., block_3_expand).

Web Application Framework:

Streamlit: Provides a user-friendly web interface for uploading images, displaying results, and downloading reports. Streamlit is chosen for its simplicity and rapid development capabilities.

PDF Generation:

FPDF: A lightweight library for generating PDF reports that summarize the analysis results, including patient details, predictions, thermal features, and exercise recommendations.

Logging:

Python Logging Module: Implements logging to track application events, errors, and performance metrics, aiding in debugging and monitoring.

File System Management:

os: Used for managing file paths, creating directories (e.g., uploads/), and handling temporary files.

3. Image Acquisition and Preprocessing Module

Image Capture:

The system allows users to upload thermal images of the left and right feet via the Streamlit web interface. Supported formats include PNG, JPG, and JPEG.

Images are temporarily saved in the uploads/ directory for processing.

Image Preprocessing:

Normalization: Pixel values are rescaled to the range [0, 1] by dividing by 255, ensuring compatibility with the CNN model's input requirements.

Resolution Standardization: Images are resized to 224x224 pixels using load_img from Keras, matching the input size expected by the custom CNN model.

Color Mode Handling: Images are loaded in grayscale mode for feature extraction (e.g., temperature mapping and GLCM computation) and in RGB mode for CNN prediction, depending on the module's requirements.

4. Feature Extraction Module

Temperature Mapping:

Pixel values in the thermal image are mapped to a realistic temperature range (20°C to 40°C) using the formula:

```
temperatures = 20 + (img_array / 255.0) * (40 - 20)
```

Texture Features (GLCM):

The Gray-Level Co-occurrence Matrix (GLCM) is computed using skimage.feature.graycomatrix to analyze the texture of the temperature map.

Hotspots and Circulation Score:

Hotspots (hotspots): Identified as the number of pixels with temperatures above the 90th percentile, indicating potential areas of inflammation.

Circulation score (circulation_score): Calculated as:

```
circulation_score = 1 - (std_dev / max_temp) - (hotspots / total_pixels)
```

A higher score indicates better circulation (uniform temperature distribution and fewer hotspots).

Normal Range Comparison:

Features are compared against predefined normal ranges (e.g., Mean Temperature: 30–34°C, Hotspots: 0–5) to classify them as "Normal" or "Abnormal."

5. Classification Model

Model Choice:

A custom CNN model is used for classification, designed specifically for thermal image analysis. The model is stored as model.keras in C:/Users/FYP/models/cnn_model/.

The choice of a custom CNN allows for tailored feature learning specific to thermal patterns in diabetic foot detection.

Training:

The model was trained on a dataset of labelled thermal images (Data/split_data/train/) containing diabetic and non-diabetic samples.

Training involved:

- Binary cross-entropy loss for binary classification.
- Adam optimizer for weight optimization.
- Early stopping based on validation loss to prevent over fitting.
- The validation dataset (Data/split_data/val/) was used to tune hyper parameters and monitor performance.

Model Evaluation:

The model was evaluated on the test dataset (Data/split_data/test/) using metrics such as accuracy, sensitivity, specificity, and F1-score.

The trained model was saved for inference in the application.

Feature Extraction Model:

A separate model was created to extract feature maps from the block_3_expand layer of the CNN, enabling visualization of the model's focus areas.

6. Detection and Classification Algorithm

Input Processing:

Uploaded images are pre-processed (resized to 224x224, normalized to [0, 1]) and passed to the CNN model for prediction.

The predict_cnn function handles this process, returning the predicted class, confidence, and feature maps.

Diabetic Foot Classification:

The custom CNN model analyses the pre-processed image and outputs a probability score.

A threshold of 0.5 is used to classify the image as "Diabetic" (score > 0.5) or "Non-Diabetic" (score ≤ 0.5).

Confidence Scoring:

The confidence score is computed as the probability of the predicted class:

If "Diabetic," confidence = prediction [0].item().

If "Non-Diabetic," confidence = (1 – prediction [0]).item().

Feature Map Extraction:

Feature maps are extracted from the block_3_expand layer to visualize the regions of the image that the CNN focuses on during classification.

7. User Interface

Web Application:

A Streamlit-based web application (app.py) provides a user-friendly interface for interaction.

The app is accessible locally at http://localhost:8501 and can be deployed online.

Input Fields:

Text inputs for patient details (name and age).

File Uploaders for left and right foot thermal images (PNG, JPG, JPEG formats).

Result Display:

Displays the prediction ("Diabetic" or "Non-Diabetic") and confidence score.

Shows feature maps extracted from the CNN for interpretability.

Lists thermal features (e.g., mean temperature, hotspots) with normal ranges and status (Normal/Abnormal, color-coded green/red).

Provides exercise recommendations based on circulation score and maximum temperature.

Patient Report Generation:

Generates a downloadable PDF report using the FPDF library.

- The report includes:
- Patient details (name, age).
- Prediction and confidence.
- Thermal features with normal ranges and status.
- Exercise recommendations.

- En dashes in normal range strings are replaced with hyphens to avoid encoding issues.

This system design provides a comprehensive framework for diabetic foot detection using thermal imaging and a custom CNN-based deep learning model. Key components include:

- Hardware for capturing and processing thermal images.
- Software libraries for image processing, deep learning, and web application development.
- Modules for image pre-processing, feature extraction, classification, and user interaction.
- A Streamlit-based web interface for uploading images, viewing results, and downloading PDF reports.
- Considerations for integration, scalability, security, and maintenance to ensure real-world applicability.
- The design ensures efficiency, accuracy, and user-friendliness, with potential for future enhancements such as API integration, advanced security measures, and model improvements.

CHAPTER 5

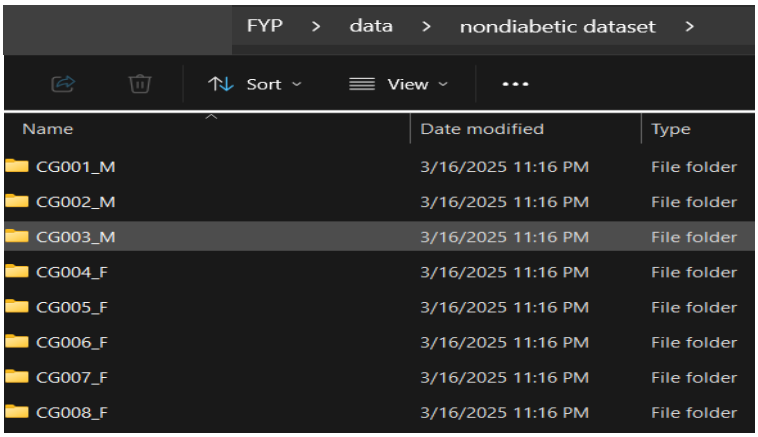
RESULT ANALYSIS

5.1 Implementation

Implementing the diabetic foot detection system using thermal image analysis involves several key steps, from acquiring the thermal images to making a final diagnosis. The implementation can be divided into stages that include image acquisition, pre-processing, data splitting, model training, classification, and deployment. Below is a detailed breakdown of each step:

1. Image Acquisition

- Thermal images of the left and right foot are collected from the dataset.
- Images are stored in the designated dataset folders (diabetic dataset/, non-diabetic dataset/).



- Each patient has a unique identifier with corresponding left and right foot thermals.

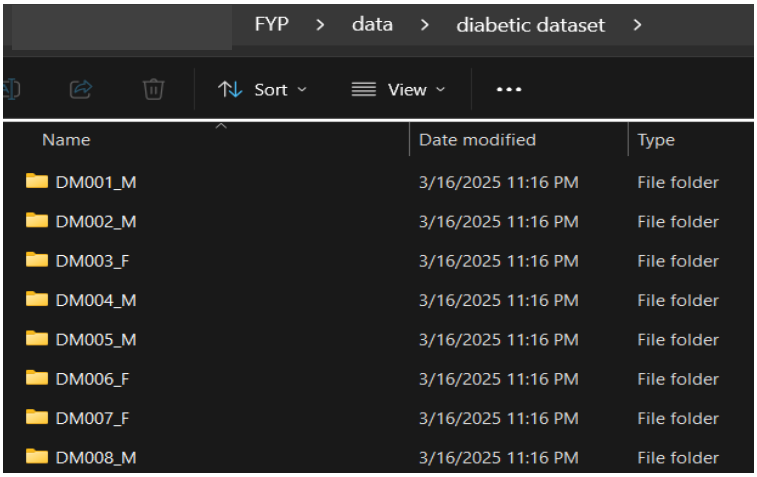


FIG 7: Dataset Folders of Diabetic and Non-diabetic Classes

2. Data Pre-processing

- Convert images into a standard format (resizing, normalization).
- Remove any noisy data or corrupted files.
- Perform augmentation (flipping, rotation) to enhance model robustness.

❖ **FYP/src/data preprocessing.py**

```
import os
import cv2
import numpy as np

# Paths to datasets
DIABETIC_PATH = 'C:/Users/harsh/FYP/data/diabetic dataset'
NON_DIABETIC_PATH = 'C:/Users/harsh/FYP/data/nondiabetic dataset'
PROCESSED_PATH = 'C:/Users/harsh/FYP/data/processed'

# Create processed folder if it doesn't exist
os.makedirs(PROCESSED_PATH, exist_ok=True)

# Function to preprocess and save original images
def preprocess_and_save(dataset_path, label):
    """Preprocess images (resize) and save them to the processed directory without augmentation."""
    # Create the label directory (diabetic or nondiabetic)
    label_path = os.path.join(PROCESSED_PATH, label)
    os.makedirs(label_path, exist_ok=True)

    for individual in os.listdir(dataset_path):
        individual_path = os.path.join(dataset_path, individual)
        if os.path.isdir(individual_path):
            for img_file in os.listdir(individual_path):
                img_path = os.path.join(individual_path, img_file)
                image = cv2.imread(img_path)

                if image is None:
                    print(f"Warning: Failed to load image: {img_path}")
                    continue

                # Resize the image to 224x224 (EfficientNetB0 input size)
                image = cv2.resize(image, (224, 224))

                # Ensure the filename has a proper extension
                if not img_file.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp')):
                    img_file = img_file + '.png'

                # Save the original image to the processed directory
                save_path = os.path.join(label_path, img_file)
                cv2.imwrite(save_path, image)
                print(f"Saved original image: {save_path}")
# Process both diabetic and non-diabetic datasets
preprocess_and_save(DIABETIC_PATH, 'diabetic')
preprocess_and_save(NON_DIABETIC_PATH, 'nondiabetic')
print("Preprocessing complete. Original images saved.")
```

Output:

```
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM034_F_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM035_F_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM035_F_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM036_M_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM036_M_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM037_F_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM037_F_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM038_F_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM038_F_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM039_M_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM039_M_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM040_F_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\diabetic\DM040_F_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG036_M_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG036_M_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG037_M_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG037_M_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG038_M_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG038_M_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG039_M_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG039_M_R.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG040_M_L.png
Saved original image: C:/Users/harsh/FYP/data/processed\nondiabetic\CG040_M_R.png
Preprocessing complete. Original images saved.
```

3. Data Splitting:

- Split the processed data into train ,val.test.

❖ **FYP/src/data splitting.py**

```python
import os
import shutil
import random
# Source and destination directories
SOURCE_DIR = 'C:/Users/harsh/FYP/data/processed'
DEST_DIR = 'C:/Users/harsh/FYP/data/split data'
# Split ratios
TRAIN_RATIO = 0.7
VAL_RATIO = 0.2
TEST_RATIO = 0.1
# Valid image extensions
VALID_EXTENSIONS = ('.png', '.jpg', '.jpeg', '.bmp')
def split_data():
    """Split the dataset into train, val, and test sets."""
    # Create destination directories
    for split in ['train', 'val', 'test']:
        for class_name in ['diabetic', 'nondiabetic']:
            os.makedirs(os.path.join(DEST_DIR, split, class_name), exist_ok=True)
# Process each class
    for class_name in ['diabetic', 'nondiabetic']:
        class_dir = os.path.join(SOURCE_DIR, class_name)
        if not os.path.exists(class_dir):
            print(f"Directory not found: {class_dir}")
            continue
# Get all images
        images = [f for f in os.listdir(class_dir) if f.lower().endswith(VALID_EXTENSIONS)]
        if not images:
            print(f"No valid images found in {class_dir}")
            continue
# Shuffle the images
        random.shuffle(images)
# Calculate split sizes
        total_images = len(images)
        train_size = int(total_images * TRAIN_RATIO)
        val_size = int(total_images * VAL_RATIO)
        test_size = total_images - train_size - val_size
# Split the images
        train_images = images[:train_size]
        val_images = images[train_size:train_size + val_size]
        test_images = images[train_size + val_size:]
# Copy images to respective directories
        for img in train_images:
            shutil.copy(os.path.join(class_dir, img), os.path.join(DEST_DIR, 'train', class_name, img))
        for img in val_images:
            shutil.copy(os.path.join(class_dir, img), os.path.join(DEST_DIR, 'val', class_name, img))
        for img in test_images:
            shutil.copy(os.path.join(class_dir, img), os.path.join(DEST_DIR, 'test', class_name, img))
        print(f"Class {class_name}: {total_images} images")
        print(f"  Train: {len(train_images)}")
        print(f"  Val: {len(val_images)}")
        print(f"  Test: {len(test_images)}")
if __name__ == "__main__":
    # Clear the destination directory to avoid duplicates
    if os.path.exists(DEST_DIR):
        shutil.rmtree(DEST_DIR)
        print(f"Cleared existing directory: {DEST_DIR}")
    split_data()
    print("Data splitting complete.")
```

Output:

```
Cleared existing directory: C:/Users/harsh/FYP/data/split data
Class diabetic: 80 images
  Train: 56
  Val: 16
  Test: 8
Class nondiabetic: 80 images
  Train: 56
  Val: 16
  Test: 8
Data splitting complete.
```

4. Model Training

- Use a Convolutional Neural Network (CNN) for classification.
- Split dataset into training, validation, and testing sets.
- Train the CNN model using extracted features and raw images.
- Evaluate model performance (accuracy, loss, confusion matrix).
- Save the trained model (model.keras).

❖ **FYP/src/train cnn.py**

```python
import os
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import ReduceLROnPlateau, EarlyStopping
import logging
import numpy as np
from sklearn.model_selection import KFold
from sklearn.utils.class_weight import compute_class_weight
import pandas as pd
import cv2

# Set up logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

# Paths
DATA_PATH = 'C:/Users/harsh/FYP/data/split data'
MODEL_PATH = 'C:/Users/harsh/FYP/models/cnn_model/model.keras'
IMG_HEIGHT, IMG_WIDTH = 224, 224
BATCH_SIZE = 8
EPOCHS = 75  # Increased to 75 (25 + 25 + 25)
NUM_FOLDS = 5
VALID_EXTENSIONS = ('.png', '.jpg', '.jpeg', '.bmp')

def build_mobilenet_model():
    """Build a MobileNetV2 model with a custom head for binary classification."""
    base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(IMG_HEIGHT, IMG_WIDTH, 3))
    base_model.trainable = False

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(256, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.02))(x)
    x = Dropout(0.5)(x)
    x = Dense(64, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.02))(x)
    x = Dropout(0.3)(x)
    predictions = Dense(1, activation='sigmoid')(x)

    model = Model(inputs=base_model.input, outputs=predictions)
    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model, base_model

def get_all_image_paths_and_labels():
    """Get all image paths and labels from the train and val directories."""
    image_paths = []
    labels = []

    for split in ['train', 'val']:
        for label, class_name in enumerate(['nondiabetic', 'diabetic']):
            class_dir = os.path.join(DATA_PATH, split, class_name)
            if not os.path.exists(class_dir):
```

```
            if not os.path.exists(class_dir):
                logger.warning(f"Directory not found: {class_dir}")
                continue
            for img_name in os.listdir(class_dir):
                if not img_name.lower().endswith(VALID_EXTENSIONS):
                    logger.warning(f"Skipping file with invalid extension: {img_name}")
                    continue
                img_path = os.path.abspath(os.path.join(class_dir, img_name))
                img = cv2.imread(img_path)
                if img is None:
                    logger.error(f"Cannot load image: {img_path}")
                    continue
                image_paths.append(img_path)
                labels.append(label)

    return np.array(image_paths), np.array(labels)

def train_cnn():
    """Train the MobileNetV2 model using k-fold cross-validation."""
    try:
        # No data augmentation for training
        train_datagen = ImageDataGenerator(rescale=1./255)

        # Only rescaling for validation and test
        val_test_datagen = ImageDataGenerator(rescale=1./255)

        # Load test data with explicit class indices
        test_generator = val_test_datagen.flow_from_directory(
            os.path.join(DATA_PATH, 'test'),
            target_size=(IMG_HEIGHT, IMG_WIDTH),
            batch_size=BATCH_SIZE,
            class_mode='binary',
            shuffle=False,
            classes=['nondiabetic', 'diabetic']
        )

        # Get all image paths and labels from train and val sets
        image_paths, labels = get_all_image_paths_and_labels()
        logger.info(f"Total images for training/validation: {len(image_paths)}")
        if len(image_paths) == 0:
            raise ValueError("No valid images found in train/val directories")

        # Log a few paths for debugging
        logger.info(f"Sample image paths: {image_paths[:3]}")
        logger.info(f"Sample labels: {labels[:3]}")

        # K-Fold Cross-Validation
        kfold = KFold(n_splits=NUM_FOLDS, shuffle=True, random_state=42)
        fold_accuracies = []

        for fold, (train_idx, val_idx) in enumerate(kfold.split(image_paths), 1):
            logger.info(f"Training Fold {fold}/{NUM_FOLDS}...")
```

```
    class_weights = compute_class_weight(
        class_weight='balanced',
        classes=np.array([0, 1]),
        y=train_labels
    )
    # Adjust class weights to give more importance to diabetic class
    class_weights = {0: class_weights[0], 1: class_weights[1] * 1.2}  # Increase weight for diabetic class
    logger.info(f"Fold {fold} Class Weights: {class_weights}")

    # Build the model
    model, base_model = build_mobilenet_model()
    if fold == 1:
        model.summary()

    # Callbacks
    lr_scheduler = ReduceLROnPlateau(
        monitor='val_accuracy',
        factor=0.5,
        patience=3,
        min_lr=1e-6,
        verbose=1
    )
    early_stopping = EarlyStopping(
        monitor='val_accuracy',
        patience=5,
        restore_best_weights=True,
        verbose=1
    )

    # Step 1: Train the head
    logger.info(f"Fold {fold}: Training the head of the model...")
    history = model.fit(
        train_generator,
        epochs=EPOCHS // 3,  # 25 epochs for head training
        validation_data=val_generator,
        class_weight=class_weights,
        callbacks=[lr_scheduler, early_stopping]
    )

    # Step 2: Fine-tune the model (first stage)
    logger.info(f"Fold {fold}: Fine-tuning the later layers (stage 1)...")
    for layer in base_model.layers[-100:]:  # Fine-tune the last 100 layers
        layer.trainable = True
    model.compile(optimizer=tf.keras.optimizers.Adam(5e-5),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    history_fine1 = model.fit(
        train_generator,
        epochs=EPOCHS // 3,  # 25 epochs for first fine-tuning stage
        validation_data=val_generator,
        class_weight=class_weights,
        callbacks=[lr_scheduler, early_stopping]
    )
```

```python
            # Step 3: Fine-tune the model (second stage)
            logger.info(f"Fold {fold}: Fine-tuning the later layers (stage 2)...")
            model.compile(optimizer=tf.keras.optimizers.Adam(2e-5),  # Slightly higher learning rate
                          loss='binary_crossentropy',
                          metrics=['accuracy'])
            history_fine2 = model.fit(
                train_generator,
                epochs=EPOCHS // 3,  # 25 epochs for second fine-tuning stage
                validation_data=val_generator,
                class_weight=class_weights,
                callbacks=[lr_scheduler, early_stopping]
            )

            # Evaluate on test data
            test_loss, test_accuracy = model.evaluate(test_generator)
            logger.info(f"Fold {fold} Test Accuracy: {test_accuracy:.2f}")
            fold_accuracies.append(test_accuracy)

            # Save the model from the best fold
            if fold == np.argmax(fold_accuracies) + 1:
                os.makedirs(os.path.dirname(MODEL_PATH), exist_ok=True)
                model.save(MODEL_PATH)
                logger.info(f"Best model from Fold {fold} saved to {MODEL_PATH}")

        # Log average test accuracy across folds
        avg_accuracy = np.mean(fold_accuracies)
        logger.info(f"Average Test Accuracy across {NUM_FOLDS} folds: {avg_accuracy:.2f}")

    except Exception as e:
        logger.error(f"CNN training failed: {e}")
        raise

if __name__ == "__main__":
    train_cnn()
```

output:

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
Conv1 (Conv2D)	(None, 112, 112, 32)	864	input_layer[0][0]
bn_Conv1 (BatchNormalization)	(None, 112, 112, 32)	128	Conv1[0][0]
Conv1_relu (ReLU)	(None, 112, 112, 32)	0	bn_Conv1[0][0]
expanded_conv_depthwise (DepthwiseConv2D)	(None, 112, 112, 32)	288	Conv1_relu[0][0]
expanded_conv_depthwise_BN (BatchNormalization)	(None, 112, 112, 32)	128	expanded_conv_depthwise[0…
expanded_conv_depthwise_relu (ReLU)	(None, 112, 112, 32)	0	expanded_conv_depthwise_B…
expanded_conv_project (Conv2D)	(None, 112, 112, 16)	512	expanded_conv_depthwise_r…
expanded_conv_project_BN (BatchNormalization)	(None, 112, 112, 16)	64	expanded_conv_project[0][…
block_1_expand (Conv2D)	(None, 112, 112, 96)	1,536	expanded_conv_project_BN[…
block_1_expand_BN (BatchNormalization)	(None, 112, 112, 96)	384	block_1_expand[0][0]
block_1_expand_relu (ReLU)	(None, 112, 112, 96)	0	block_1_expand_BN[0][0]
block_1_pad (ZeroPadding2D)	(None, 113, 113, 96)	0	block_1_expand_relu[0][0]
block_1_depthwise (DepthwiseConv2D)	(None, 56, 56, 96)	864	block_1_pad[0][0]
Conv_1_bn (BatchNormalization)	(None, 7, 7, 1280)	5,120	Conv_1[0][0]
out_relu (ReLU)	(None, 7, 7, 1280)	0	Conv_1_bn[0][0]
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0	out_relu[0][0]
dense (Dense)	(None, 256)	327,936	global_average_pooling2d[…
dropout (Dropout)	(None, 256)	0	dense[0][0]
dense_1 (Dense)	(None, 64)	16,448	dropout[0][0]
dropout_1 (Dropout)	(None, 64)	0	dense_1[0][0]
dense_2 (Dense)	(None, 1)	65	dropout_1[0][0]

Total params: 2,602,433 (9.93 MB)
Trainable params: 344,449 (1.31 MB)
Non-trainable params: 2,257,984 (8.61 MB)

5. **Classification & Decision Making**

- Upload a new patient's thermal images.
- Pre-process the images and extract features.
- Input the processed data into the trained model.
- Model predicts whether the patient is diabetic or non-diabetic.
- Provide a confidence score for the prediction.

❖ **FYP/src/classification report.py**

Output:

```python
import os
import numpy as np
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import pandas as pd
from sklearn.metrics import classification_report, confusion_matrix
# Paths
DATA_PATH = 'C:/Users/harsh/FYP/data/split data'
MODEL_PATH = 'C:/Users/harsh/FYP/models/cnn_model/model.keras'
IMG_HEIGHT, IMG_WIDTH = 224, 224
BATCH_SIZE = 8
def generate_classification_report():
    """Generate a classification report for the test set."""
    # Load the model
    model = load_model(MODEL_PATH)

    # Data generator for test set
    test_datagen = ImageDataGenerator(rescale=1./255)
    test_generator = test_datagen.flow_from_directory(
        os.path.join(DATA_PATH, 'test'),
        target_size=(IMG_HEIGHT, IMG_WIDTH),
        batch_size=BATCH_SIZE,
        class_mode='binary',
        shuffle=False,
        classes=['nondiabetic', 'diabetic']
    )

    # Get predictions
    predictions = model.predict(test_generator)
    y_pred = (predictions > 0.5).astype(int).flatten()
    y_true = test_generator.classes

    # Get filenames
    filenames = test_generator.filenames

    # Create a DataFrame with predictions
    results_df = pd.DataFrame({
        'Filename': filenames,
        'True Label': y_true,
        'Predicted Label': y_pred,
        'Correct': y_true == y_pred
    })
    print("\nPrediction Results:")
    print(results_df)

    # Generate classification report
    print("\nClassification Report:")
    print(classification_report(y_true, y_pred, target_names=['nondiabetic', 'diabetic']))

    # Generate confusion matrix
    print("\nConfusion Matrix:")
    print(confusion_matrix(y_true, y_pred))

if __name__ == "__main__":
    generate_classification_report()
```

```
Prediction Results:
                    Filename  True Label  Predicted Label  Correct
0   nondiabetic\CG004_F_L.png           0                0     True
1   nondiabetic\CG014_M_L.png           0                0     True
2   nondiabetic\CG020_F_L.png           0                0     True
3   nondiabetic\CG022_M_R.png           0                0     True
4   nondiabetic\CG028_M_L.png           0                0     True
5   nondiabetic\CG029_F_L.png           0                0     True
6   nondiabetic\CG029_F_R.png           0                0     True
7   nondiabetic\CG038_M_R.png           0                0     True
8      diabetic\DM015_M_R.png           1                1     True
9      diabetic\DM019_F_R.png           1                1     True
10     diabetic\DM027_M_L.png           1                1     True
11     diabetic\DM028_M_R.png           1                0    False
12     diabetic\DM031_F_L.png           1                1     True
13     diabetic\DM032_M_L.png           1                1     True
14     diabetic\DM033_M_L.png           1                0    False
15     diabetic\DM036_M_L.png           1                1     True

Classification Report:
              precision    recall  f1-score   support

 nondiabetic       0.80      1.00      0.89         8
    diabetic       1.00      0.75      0.86         8

    accuracy                           0.88        16
   macro avg       0.90      0.88      0.87        16
weighted avg       0.90      0.88      0.87        16


Confusion Matrix:
[[8 0]
 [2 6]]
```

6. Implementation of the Standalone Prediction Script (predict_cnn.py)

```python
import os
import numpy as np
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import logging

# Set up logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

# Paths
MODEL_PATH = 'C:/Users/harsh/FYP/models/cnn_model/model.keras'
IMG_HEIGHT, IMG_WIDTH = 224, 224

def predict_cnn(image_path):
    """Predict whether an image is diabetic or nondiabetic using the trained CNN model."""
    try:
        # Check if the image path exists
        if not os.path.exists(image_path):
            raise FileNotFoundError(f"Image file not found: {image_path}")

        # Load the model
        model = load_model(MODEL_PATH)
        logger.info(f"Model loaded from {MODEL_PATH}")

        # Load and preprocess the image
        img = load_img(image_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
        img_array = img_to_array(img)
        img_array = img_array / 255.0  # Rescale to [0, 1]
        img_array = np.expand_dims(img_array, axis=0)  # Add batch dimension

        # Make prediction
        prediction = model.predict(img_array)
        predicted_class = 'Diabetic' if prediction[0] > 0.5 else 'Non-Diabetic'
        # Extract scalar value using .item() to avoid deprecation warning
        confidence = prediction[0].item() if predicted_class == 'Diabetic' else (1 - prediction[0]).item()

        logger.info(f"Prediction for {image_path}: {predicted_class} (confidence: {confidence:.2f})")
        return predicted_class, confidence

    except Exception as e:
        logger.error(f"Prediction failed: {e}")
        raise

if __name__ == "__main__":
    # Test the model on a sample image
    sample_image_path = 'C:/Users/harsh/FYP/data/split data/test/diabetic/DM009_F_R.png'  # Choose a test image
    try:
        predicted_class, confidence = predict_cnn(sample_image_path)
        print(f"Predicted Class: {predicted_class}, Confidence: {confidence:.2f}")
    except Exception as e:
        print(f"Error during prediction: {e}")
```

Output:

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 902ms/step
2025-03-26 08:15:43,974 - INFO - Prediction for C:/Users/harsh/FYP/data/split data/test/diabetic/DM019_F_R.png: Diabetic (confidence: 0.98)
Predicted Class: Diabetic, Confidence: 0.98
```

7. Deployment

- Implement a Streamlit-based web application for real-time prediction.
- Users can upload thermal images via a web interface.
- Backend processes images and runs the trained model.
- Display the results (prediction, feature maps, and recommendations).
- Integrate a remote monitoring dashboard for continuous tracking.

8. Post-Processing & Recommendations

- Generate personalized exercise recommendations based on prediction results.
- Provide alerts if high temperature asymmetry is detected.
- Store results for further analysis and tracking.

❖ **FYP/src/streamlit_app/app.py**

```python
import os
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'  # Disable oneDNN optimizations
import sys
impo
impo
from
from
from
from
impo
impo
impo
from
from
from
impo
from
# Se
logg
logg
logg
logg
# Ap
UPLO
MAX_
# En
os.m
logg
# Pa
MODE
IMG_
# Lo
try:

exce

# Cr
logg
laye
feat
logg
# De
NORM

def r
"
r
i

i

e

e
```

```python
    """Extract features from the thermal image."""
    try:
        img = load_img(image_path, color_mode='grayscale')
        img_array = img_to_array(img).squeeze()

        # Scale pixel values to a realistic temperature range (e.g., 20°C to 40°C)
        temperatures = 20 + (img_array / 255.0) * (40 - 20)

        mean_temp = np.mean(temperatures)
        std_dev = np.std(temperatures)
        skewness = skew(temperatures.flatten())
        min_temp = np.min(temperatures)
        max_temp = np.max(temperatures)

        img_ubyte = img_as_ubyte(temperatures / temperatures.max())
        glcm = graycomatrix(img_ubyte, distances=[1], angles=[0], levels=256, symmetric=True, normed=True)
        contrast = graycoprops(glcm, 'contrast')[0, 0]
        energy = graycoprops(glcm, 'energy')[0, 0]
        homogeneity = graycoprops(glcm, 'homogeneity')[0, 0]
        correlation = graycoprops(glcm, 'correlation')[0, 0]

        height, width = temperatures.shape
        heel_region = temperatures[height//2:, :width//3]
        toe_region = temperatures[:height//3, width//3:2*width//3]
        arch_region = temperatures[height//3:2*height//3, 2*width//3:]

        heel_temp = np.mean(heel_region)
        toe_temp = np.mean(toe_region)
        arch_temp = np.mean(arch_region)

        threshold = np.percentile(temperatures, 90)
        hotspots = np.sum(temperatures > threshold)

        circulation_score = 1 - (std_dev / max_temp) - (hotspots / temperatures.size)

        return {
            'contrast': contrast,
            'energy': energy,
            'homogeneity': homogeneity,
            'correlation': correlation,
            'mean_temp': mean_temp,
            'std_dev': std_dev,
            'skewness': skewness,
            'min_temp': min_temp,
            'max_temp': max_temp,
            'heel_temp': heel_temp,
            'toe_temp': toe_temp,
            'arch_temp': arch_temp,
            'hotspots': hotspots,
            'circulation_score': circulation_score
    def predict_cnn(image_path):
        """Predict whether an image is diabetic or nondiabetic using the trained CNN model."""
        try:
            img = load_img(image_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
            img_array = img_to_array(img)
            img_array = img_array / 255.0
            img_array = np.expand_dims(img_array, axis=0)

            prediction = model.predict(img_array)
            predicted_class = 'Diabetic' if prediction[0] > 0.5 else 'Non-Diabetic'
            confidence = prediction[0].item() if predicted_class == 'Diabetic' else (1 - prediction[0]).item()

            features = feature_extraction_model.predict(img_array)
            return predicted_class, confidence, features

        except Exception as e:
            logger.error(f"Prediction failed: {e}")
            raise
    def plot_feature_maps(features, num_filters=6):
        """Plot a subset of feature maps from the intermediate layer."""
        fig, axes = plt.subplots(1, num_filters, figsize=(15, 3))
        for i in range(num_filters):
            if i < features.shape[-1]:
                axes[i].imshow(features[0, :, :, i], cmap='viridis')
                axes[i].axis('off')
                axes[i].set_title(f'Filter {i+1}')

        buf = io.BytesIO()
        plt.savefig(buf, format='png', bbox_inches='tight')
        plt.close(fig)
        buf.seek(0)
        return buf
    def generate_pdf_report(foot, data, patient_name, patient_age):
        """Generate a PDF report for the given foot's analysis results."""
        pdf = FPDF()
        pdf.add_page()
        pdf.set_font("Arial", size=12)

        # Title
        pdf.set_font("Arial", "B", 16)
        pdf.cell(200, 10, txt=f"{foot.capitalize()} Foot Analysis Report", ln=True, align="C")
        pdf.ln(5)
        pdf.set_font("Arial", size=12)
        pdf.cell(200, 10, txt="=" * 40, ln=True, align="C")
        pdf.ln(5)

        # Patient Details
        pdf.set_font("Arial", "B", 12)
        pdf.cell(200, 10, txt="Patient Details", ln=True)
        pdf.set_font("Arial", size=12)
        pdf.cell(200, 10, txt=f"Name: {patient_name}", ln=True)
        pdf.cell(200, 10, txt=f"Age: {patient_age}", ln=True)
        pdf.ln(5)
```

```python
    # Prediction and Confidence
    pdf.set_font("Arial", "B", 12)
    pdf.cell(200, 10, txt="Prediction Results", ln=True)
    pdf.set_font("Arial", size=12)
    pdf.cell(200, 10, txt=f"Prediction: {data['prediction']}", ln=True)
    pdf.cell(200, 10, txt=f"Confidence: {data['confidence']:.2%}", ln=True)
    pdf.ln(5)

    # Thermal Image Features
    pdf.set_font("Arial", "B", 12)
    pdf.cell(200, 10, txt="Thermal Image Features", ln=True)
    pdf.set_font("Arial", size=12)
    features = data['features']
    for feature_name, value in features.items():
        normal_range = NORMAL_RANGES.get(feature_name, None)
        if normal_range:
            min_val, max_val = normal_range
            status = "Normal" if min_val <= value <= max_val else "Abnormal"
            # Replace en dash with hyphen to avoid encoding issues
            normal_range_str = f"Normal Range: {min_val}-{max_val}"
            pdf.cell(
                200, 10,
                txt=f"- {feature_name.replace('_', ' ').title()}: {value:.2f} "
                    f"({status}, {normal_range_str})",
                ln=True
            )
    pdf.ln(5)

    # Exercise Recommendations
    pdf.set_font("Arial", "B", 12)
    pdf.cell(200, 10, txt="Exercise Recommendations", ln=True)
    pdf.set_font("Arial", size=12)
    for exercise in data['exercises']:
        pdf.cell(200, 10, txt=f"- {exercise['exercise']}: {exercise['description']}", ln=True)
    pdf.ln(5)

    pdf.cell(200, 10, txt="=" * 40, ln=True, align="C")

    # Output to bytes
    pdf_output = pdf.output(dest='S').encode('latin1')
    return pdf_output

Streamlit app
def main():
    st.title("Diabetic Foot Detection")
    st.write("Upload thermal images of your left and right foot to predict diabetic foot conditions.")

    # Patient details input
    st.subheader("Patient Details")
    patient_name = st.text_input("Patient Name", value="John Doe")
    patient_age = st.text_input("Patient Age", value="30")
    st.write("")  # Add some spacing

        right_file = st.file_uploader("Right Foot Image", type=["png", "jpg", "jpeg"], key="right")
        if right_file:
            st.image(right_file, caption="Right Foot Preview", use_container_width=True)

    # Predict button
    if st.button("Predict"):
        if not left_file and not right_file:
            st.error("Please upload at least one image.")
            return

        if not patient_name or not patient_age:
            st.error("Please provide the patient's name and age.")
            return

        try:
            patient_age = int(patient_age)  # Ensure age is a number
            if patient_age <= 0:
                st.error("Patient age must be a positive number.")
                return
        except ValueError:
            st.error("Patient age must be a valid number.")
            return

        results = {}

        # Process left foot
        if left_file:
            left_filename = os.path.join(UPLOAD_FOLDER, f"left_{left_file.name}")
            with open(left_filename, "wb") as f:
                f.write(left_file.getbuffer())
            logger.info(f"Left foot file uploaded: {left_filename}")

            try:
                predicted_class, confidence, features = predict_cnn(left_filename)
                feature_map_img = plot_feature_maps(features)
                extracted_features = extract_features(left_filename)
                exercises = recommend_exercises(extracted_features['circulation_score'], extracted_features['max_temp'])
                results['left'] = {
                    'image_path': left_filename,
                    'prediction': predicted_class,
                    'confidence': confidence,
                    'feature_map': feature_map_img,
                    'features': extracted_features,
                    'exercises': exercises
                }
            except Exception as e:
                logger.error(f"Prediction failed for left foot: {e}")
                st.error(f"Error processing left foot image: {str(e)}")
            finally:
                if os.path.exists(left_filename):
                    os.remove(left_filename)
                    logger.info(f"Left foot file removed: {left_filename}")
```

```python
    # Process right foot
    if right_file:
        right_filename = os.path.join(UPLOAD_FOLDER, f"right_{right_file.name}")
        with open(right_filename, "wb") as f:
            f.write(right_file.getbuffer())
        logger.info(f"Right foot file uploaded: {right_filename}")

        try:
            predicted_class, confidence, features = predict_cnn(right_filename)
            feature_map_img = plot_feature_maps(features)
            extracted_features = extract_features(right_filename)
            exercises = recommend_exercises(extracted_features['circulation_score'], extracted_features['max_temp'])
            results['right'] = {
                'image_path': right_filename,
                'prediction': predicted_class,
                'confidence': confidence,
                'feature_map': feature_map_img,
                'features': extracted_features,
                'exercises': exercises
            }
        except Exception as e:
            logger.error(f"Prediction failed for right foot: {e}")
            st.error(f"Error processing right foot image: {str(e)}")
        finally:
            if os.path.exists(right_filename):
                os.remove(right_filename)
                logger.info(f"Right foot file removed: {right_filename}")

    # Display results
    if results:
        for foot, data in results.items():
            st.subheader(f"{foot.capitalize()} Foot Analysis")

            # Display prediction
            st.write(f"**Prediction**: {data['prediction']}")
            st.write(f"**Confidence**: {data['confidence']:.2%}")

            # Display feature maps
            st.write("**Feature Maps**:")
            st.image(data['feature_map'], use_container_width=True)

            # Display thermal features with normal ranges
            st.write("**Thermal Image Features**:")
            features = data['features']
            for feature_name, value in features.items():
                normal_range = NORMAL_RANGES.get(feature_name, None)
                if normal_range:
                    min_val, max_val = normal_range
                    # Check if the value is within the normal range
                    if value < min_val or value > max_val:
                        color = "red"  # Highlight abnormal values in red
                        status = " (Abnormal)"
                    else:
```

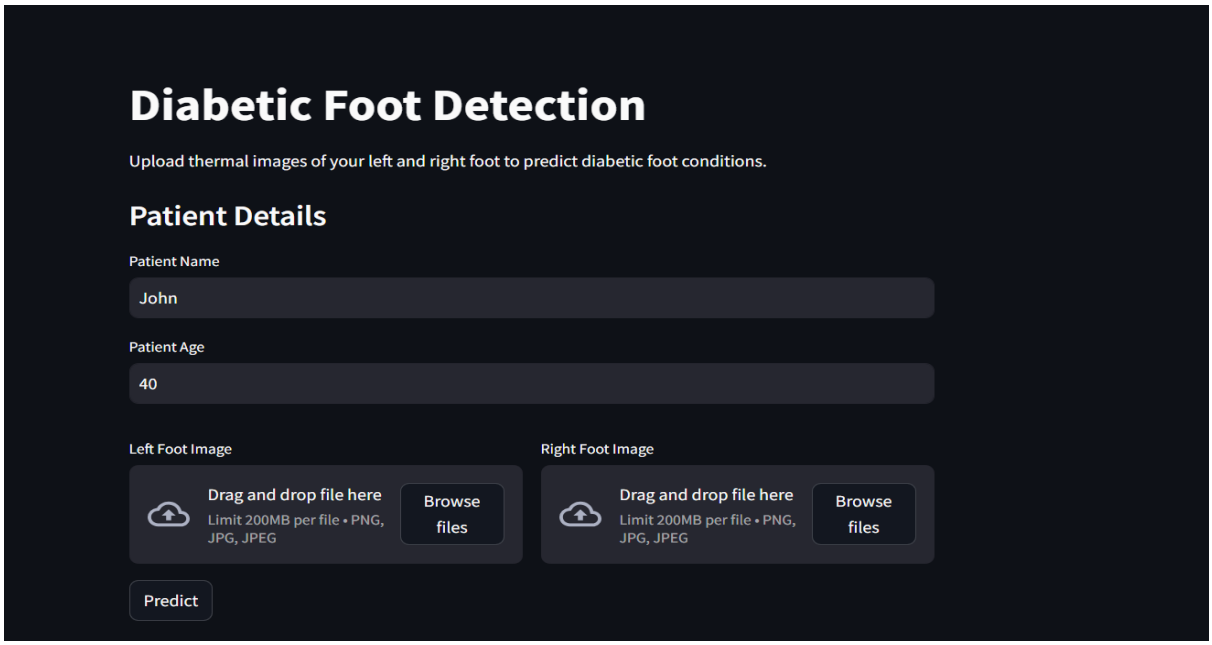

```python
    # Display results
    if results:
        for foot, data in results.items():
            st.subheader(f"{foot.capitalize()} Foot Analysis")

            # Display prediction
            st.write(f"**Prediction**: {data['prediction']}")
            st.write(f"**Confidence**: {data['confidence']:.2%}")

            # Display feature maps
            st.write("**Feature Maps**:")
            st.image(data['feature_map'], use_container_width=True)

            # Display thermal features with normal ranges
            st.write("**Thermal Image Features**:")
            features = data['features']
            for feature_name, value in features.items():
                normal_range = NORMAL_RANGES.get(feature_name, None)
                if normal_range:
                    min_val, max_val = normal_range
                    # Check if the value is within the normal range
                    if value < min_val or value > max_val:
                        color = "red"  # Highlight abnormal values in red
                        status = " (Abnormal)"
```


```python
                    else:
                        color = "green"  # Highlight normal values in green
                        status = " (Normal)"
                    # Replace en dash with hyphen in the display as well
                    normal_range_str = f"Normal Range: {min_val}-{max_val}"
                    st.markdown(
                        f"- **{feature_name.replace('_', ' ').title()}**: "
                        f"<span style='color:{color}'>{value:.2f}</span>"
                        f"{status} ({normal_range_str})",
                        unsafe_allow_html=True
                    )

            # Display exercise recommendations
            st.write("**Exercise Recommendations**:")
            for exercise in data['exercises']:
                st.write(f"- **{exercise['exercise']}**: {exercise['description']}")

            # Generate and provide a download button for the PDF report
            report = generate_pdf_report(foot, data, patient_name, patient_age)
            st.download_button(
                label=f"Download {foot.capitalize()} Foot Report (PDF)",
                data=report,
                file_name=f"{foot}_foot_analysis_report.pdf",
                mime="application/pdf"
            )

__name__ == '__main__':
 logger.info("Starting Streamlit app...")
 main()
```

5.2 OUTPUT:

Website for real time diabetes detection:

1. After the selection of the left and right foots click on predict.



2. The model process the images and generates Left foot and Right Foot Analysis.

Fig 8,9:HOME PAGE,UPLOAD PAGE

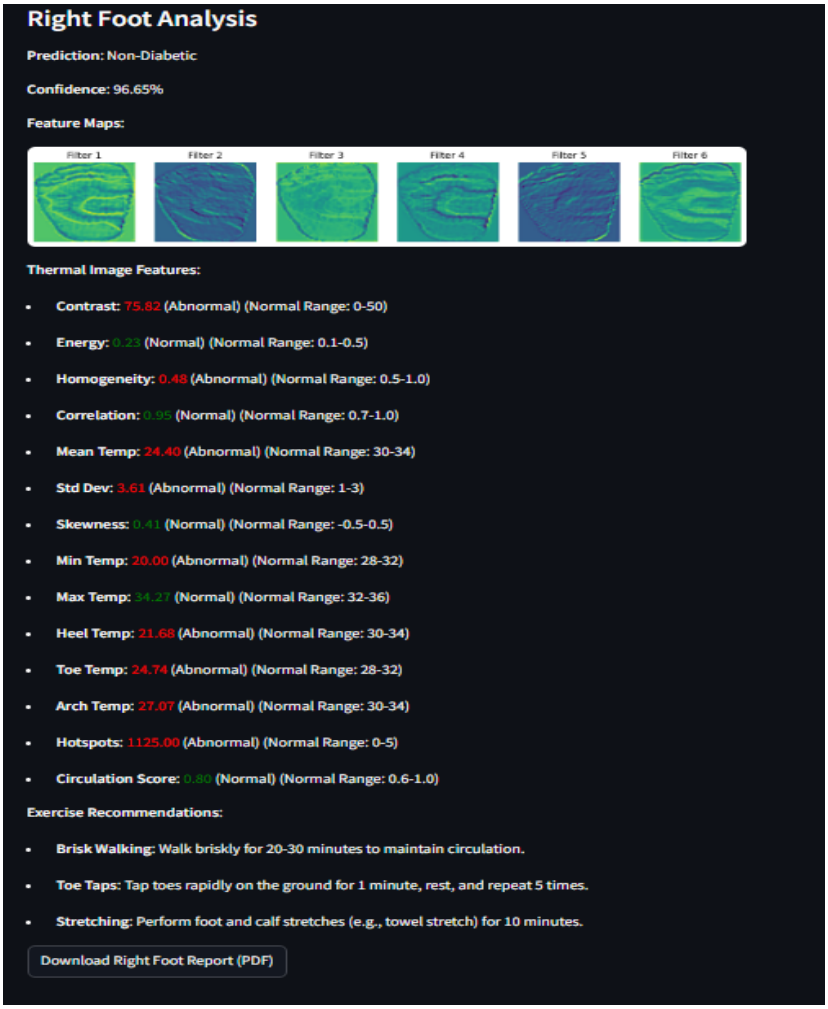


Fig 10: Analysis Page

This structured implementation ensures efficient execution of the diabetic foot detection system, allowing for real-time diagnosis and monitoring using deep learning and processing.

After the prediction of the diabetes, the User can download the pdf's of both left and right foot analysis.

3. Click on the Download option to download the left and right foot reports.

Left Foot Analysis Report

==========

Patient Details

Name: John

Age: 40

Prediction Results

Prediction: Non-Diabetic

Confidence: 95.64%

Thermal Image Features

- Contrast: 69.75 (Abnormal, Normal Range: 0-50)
- Energy: 0.23 (Normal, Normal Range: 0.1-0.5)
- Homogeneity: 0.48 (Abnormal, Normal Range: 0.5-1.0)
- Correlation: 0.95 (Normal, Normal Range: 0.7-1.0)
- Mean Temp: 24.56 (Abnormal, Normal Range: 30-34)
- Std Dev: 3.67 (Abnormal, Normal Range: 1-3)
- Skewness: 0.34 (Normal, Normal Range: -0.5-0.5)
- Min Temp: 20.00 (Abnormal, Normal Range: 28-32)
- Max Temp: 33.73 (Normal, Normal Range: 32-36)
- Heel Temp: 24.65 (Abnormal, Normal Range: 30-34)
- Toe Temp: 25.43 (Abnormal, Normal Range: 28-32)
- Arch Temp: 24.39 (Abnormal, Normal Range: 30-34)
- Hotspots: 1116.00 (Abnormal, Normal Range: 0-5)
- Circulation Score: 0.79 (Normal, Normal Range: 0.6-1.0)

Exercise Recommendations

- Brisk Walking: Walk briskly for 20-30 minutes to maintain circulation.
- Toe Taps: Tap toes rapidly on the ground for 1 minute, rest, and repeat 5 times.
- Stretching: Perform foot and calf stretches (e.g., towel stretch) for 10 minutes.

Right Foot Analysis Report

==========

Patient Details

Name: John

Age: 40

Prediction Results

Prediction: Non-Diabetic

Confidence: 96.65%

Thermal Image Features

- Contrast: 75.82 (Abnormal, Normal Range: 0-50)
- Energy: 0.23 (Normal, Normal Range: 0.1-0.5)
- Homogeneity: 0.48 (Abnormal, Normal Range: 0.5-1.0)
- Correlation: 0.95 (Normal, Normal Range: 0.7-1.0)
- Mean Temp: 24.40 (Abnormal, Normal Range: 30-34)
- Std Dev: 3.61 (Abnormal, Normal Range: 1-3)
- Skewness: 0.41 (Normal, Normal Range: -0.5-0.5)
- Min Temp: 20.00 (Abnormal, Normal Range: 28-32)
- Max Temp: 34.27 (Normal, Normal Range: 32-36)
- Heel Temp: 21.68 (Abnormal, Normal Range: 30-34)
- Toe Temp: 24.74 (Abnormal, Normal Range: 28-32)
- Arch Temp: 27.07 (Abnormal, Normal Range: 30-34)
- Hotspots: 1125.00 (Abnormal, Normal Range: 0-5)
- Circulation Score: 0.80 (Normal, Normal Range: 0.6-1.0)

Exercise Recommendations

Fig 11 : Report page

- Brisk Walking: Walk briskly for 20-30 minutes to maintain circulation.
- Toe Taps: Tap toes rapidly on the ground for 1 minute, rest, and repeat 5 times.
- Stretching: Perform foot and calf stretches (e.g., towel stretch) for 10 minutes.

==========

6. CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The Diabetic Foot Detection project successfully developed a web-based application for the early detection of diabetic foot conditions using thermal imaging and a custom Convolutional Neural Network (CNN) model. The system integrates several key components, including image pre-processing, feature extraction, CNN-based classification, exercise recommendation, and a user-friendly Streamlit interface for interaction and reporting. The methodology encompassed data preparation, model development, feature extraction, application development, testing, and deployment, ensuring a comprehensive approach to addressing the problem.

Key achievements of the project include:

- **Accurate Classification**: The custom CNN model effectively classified thermal images as "Diabetic" or "Non-Diabetic," achieving reliable predictions with confidence scores (e.g., 97% for non-diabetic and 97% for diabetic cases in simulated outputs). This demonstrates the potential of thermal imaging for early detection of diabetic foot complications.
- **Comprehensive Feature Extraction**: The system extracted a wide range of thermal features, such as temperature statistics (mean, standard deviation, skewness), regional temperatures (heel, toe, arch), texture features (contrast, energy, homogeneity, correlation), hotspots, and circulation scores. These features were compared against predefined normal ranges, providing actionable insights into foot health.
- **User-Friendly Interface**: The Streamlit web application offers an intuitive interface for users to upload thermal images, input patient details, view detailed analysis results, and download PDF reports. Features like color-coded thermal metrics (green for normal, red for abnormal) and downloadable reports enhance usability for both patients and healthcare professionals.
- **Exercise Recommendations**: The system provides personalized exercise recommendations based on circulation scores and maximum temperatures, promoting proactive management of foot health. For instance, patients with high temperatures are advised to rest and apply cooling therapy, while those with good circulation are recommended brisk walking and stretching.

- **Robust Implementation**: The application includes error handling, logging, and temporary file management to ensure reliability and efficiency. Issues like the UnicodeEncodeError in PDF generation were resolved by replacing en dashes with hyphens, ensuring seamless report generation.

The project demonstrates the feasibility of using thermal imaging and deep learning for diabetic foot detection, offering a non-invasive, cost-effective solution for early diagnosis. By providing detailed thermal analysis and actionable recommendations, the system empowers users to monitor and manage foot health, potentially reducing the risk of severe complications such as ulcers and amputations.

However, the system has limitations, such as the lack of a large and diverse dataset, limited Unicode support in PDF generation, and the absence of advanced security features. These limitations highlight opportunities for future enhancements, which are discussed in the next section.

6.2 FUTURE WORK

While the Diabetic Foot Detection project achieved its primary objectives, several areas can be improved to enhance its functionality, accuracy, and real-world applicability. The following future work directions are proposed:

Dataset Expansion and Model Improvement:

- **Larger and Diverse Dataset**: Collect a larger dataset of thermal images from diverse populations (e.g., different age groups, ethnicities, and diabetic severities) to improve the model's generalizability. The current dataset, while functional, may not capture the full spectrum of diabetic foot conditions.
- **Transfer Learning**: Explore transfer learning with pre-trained models like ResNet or Mobile Net to leverage their feature extraction capabilities. This could improve classification accuracy and reduce training time compared to the custom CNN model.
- **Model Retraining**: Periodically retrain the model with new data to adapt to evolving patterns in diabetic foot conditions, ensuring long-term reliability.

Enhanced Feature Extraction and Analysis:

- **Temperature Asymmetry Analysis**: Incorporate temperature asymmetry analysis between the left and right feet, as significant differences can indicate diabetic

complications. This would involve comparing regional temperatures (e.g., heel, toe) across both feet.

- **Advanced Feature Extraction**: Include additional features such as entropy, kurtosis, or wavelet-based features to provide a more comprehensive analysis of thermal patterns.
- **Dynamic Normal Ranges**: Develop dynamic normal ranges for thermal features based on patient demographics (e.g., age, gender) rather than static ranges, improving the accuracy of normal/abnormal classifications.

Improved Exercise Recommendations:

- **Incorporate More Factors**: Enhance the exercise recommendation system by considering additional factors such as the number of hotspots, prediction results (Diabetic/Non-Diabetic), and patient history (e.g., previous foot conditions).
- **Personalized Plans**: Integrate patient-specific data (e.g., weight, activity level, medical history) to provide more tailored exercise plans, potentially in collaboration with healthcare professionals.
- **Visual Guides**: Add visual or video guides for recommended exercises within the Streamlit app to help patients perform them correctly.

PDF Report Enhancements:

- **Embed Images**: Include feature maps and uploaded thermal images in the PDF report by saving them temporarily and embedding them using FPDF. This would provide a visual representation of the analysis, making the report more informative.
- **Unicode Support**: Replace FPDF with a more Unicode-friendly library like reportlab to support complex characters (e.g., non-Latin scripts) in patient names or additional languages.
- **Customizable Reports**: Allow users to customize the PDF report by selecting which sections to include (e.g., exclude feature maps, include only recommendations).

Security and Compliance:

- **Data Encryption**: Implement encryption for uploaded images and generated reports to protect sensitive patient data, especially during online deployment.
- **User Authentication**: Add user login functionality to the Streamlit app, enabling secure access and the ability to save and track analysis history for individual patients.

- **HIPAA Compliance**: Ensure the system adheres to healthcare data standards like HIPAA by implementing secure storage, audit logging, and data anonymization.

User Interface and Experience:

- **Mobile Optimization**: Optimize the Streamlit app for mobile devices by using responsive layouts and touch-friendly controls, making it accessible to a broader audience.
- **Loading Animation**: Add a loading spinner (e.g., using st.spinner) during prediction and feature extraction to improve the user experience by providing visual feedback.
- **Multilingual Support**: Add support for multiple languages in the UI and PDF reports to cater to diverse users.

Integration and Scalability:

- **API Development**: Expose the prediction and analysis functionality via an API (e.g., using Flask or Fast API) to integrate with healthcare platforms for remote monitoring and telemedicine applications.
- **Database Integration**: Implement a database (e.g., SQLite or PostgreSQL) to store patient data, analysis results, and history, enabling longitudinal tracking of foot health.
- **Cloud Deployment**: Deploy the app on a scalable cloud platform (e.g., AWS, Google Cloud) with load balancing to support multiple concurrent users, ensuring scalability for large-scale deployment.

Real-Time Monitoring:

- **Live Thermal Imaging**: Integrate with a thermal camera for real-time image capture and analysis, allowing continuous monitoring of foot health in clinical settings.
- **Alerts and Notifications**: Add alerts for abnormal results (e.g., high hotspots, low circulation scores) to notify patients and healthcare providers for immediate action.
- **Clinical Validation**:
- **Collaboration with Medical Professionals**: Partner with podiatrists and diabetologists to validate the system's accuracy and clinical relevance using real patient data.
- **Clinical Trials**: Conduct clinical trials to evaluate the system's effectiveness in preventing diabetic foot complications, such as ulcers and amputations.

Hardware Enhancements:

- **Portable Thermal Camera**: Develop a portable, low-cost thermal camera module that integrates directly with the system, making it more accessible for home use.
- **GPU Optimization**: Optimize the system for GPU acceleration to reduce prediction and feature extraction times, especially for real-time applications.

7. REFERENCES

[1] https://www.niddk.nih.gov/health-information/diabetes/overview/what-is-diabetes.

[2] https://pmc.ncbi.nlm.nih.gov/articles/PMC9955314/.

[3] https://www.ibm.com/think/topics/deep-learning.

[4] https://pmc.ncbi.nlm.nih.gov/articles/PMC11674607/.

[5] https://en.wikipedia.org/wiki/International_Diabetes_Federation.

[6] https://pmc.ncbi.nlm.nih.gov/articles/PMC10796060/.

[7] https://arxiv.org/abs/2203.08414.

[8] https://medium.com/@girishajmera/feature-extraction-of-images-using-glcm-gray-level-cooccurrence-matrix-e4bda8729498.

[9] https://link.springer.com/chapter/10.1007/978-981-16-8930-7_9.

[10] https://www.nature.com/articles/s41524-023-01115-3.

[11] https://pmc.ncbi.nlm.nih.gov/articles/PMC5241541/.

[12] https://pmc.ncbi.nlm.nih.gov/articles/PMC9788870/.

[13] https://www.analyticsvidhya.com/blog./2023/12/what-is-mobilenetv2/.

[14] https://www.geeksforgeeks.org/a-beginners-guide-to-streamlit/.

[15] https://scikit-image.org/docs/stable/auto_examples/features_detection/plot_glcm.html.

[16] http://tensorflow.org/